PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

TRAINEE DATA ANALYST

AN INTERNSHIP REPORT

Submitted by

Chirag BA - 20221CSE0314

Under the guidance of,

Mr. H. Senthil Kumar

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026

PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report **"Data Analyst Intern"** is a bonafide work of **Chirag BA (20221CSE0314),** who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING during 2025-2026.

Mr. H. Senthil Kumar
Internship Guide
PSCS
Presidency University

Mr. Sakthi S
Program Internship Coordinator
PSCS
Presidency University

Dr. Md Ziaur Rahman
School Internship Coordinator
PSCS
Presidency University

Dr. Asif Mohamed H B
Head of the Department
PSCS
Presidency University

Dr. Duraipandian N
Dean
PSCS / PSIS
Presidency University

Sl.No	Name of the Examiner	Signature

PRESIDENCY UNIVERSITY

PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named **Chirag BA**, hereby declare that the internship work titled **"Data Analyst Intern"** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE and ENGINEERING during the academic year of 2025-2026. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Chirag BA USN: 20221CSE0314 Signature

PLACE: BENGALURU

DATE: 22 - April 2026

INTERNSHIP COMPLETION CERTIFICATE

NINELEAPS

Intern ID: NLI-747 Date: 6th January 2026

Chirag B A
No 318, Mahaveer Rich Apartment, AGB
Layout, Chikkasandra, Bengaluru, 560090

Subject: Offer of Employment

Dear Chirag ,

Congratulations! Further to your application for employment with us, and the subsequent selection process, we are delighted to offer you a position as Junior Data Analyst with Nineleaps Technology Solutions Private Limited (the "Company"). The designation will be subject to your performance post the training period. Your location of reporting is Bangalore (this may be subject to change), India and the scheduled date of your joining will be 19th January 2026.

Offer would stand automatically canceled in case of any deviations in information or if you fail to report on or before the agreed date.

Your CTC will be Rs. 6,60,000/- per annum (Six Lakh Sixty Thousand Rupees Only). This CTC includes the fixed and variable component as detailed in the Compensation Details sheet provided under **Annexure**

Your appointment will be governed by the terms and conditions of employment that you shall sign at the time of joining. Any breach of the same may result in withdrawal of this offer and or termination of your employment.

Please sign the duplicate of this offer as your acceptance and forward the same to us. If you do not do so within seven days after the date of this letter, the offer will lapse.

Welcome to the Nineleaps family! We look forward to your joining us and wish you a long and fruitful career with the Company.

Yours Sincerely,
for Nineleaps Technology Solutions Private Ltd.

Abhinaya K
Human Resource

I have read, understood, and accept the terms and conditions and will join on 19/01/2026.

Signature: Date: 06/01/2026

NINELEAPS

ANNEXURE A

The terms & conditions of our offer are as follows:

1. Training Period:

You will undergo a training period of 6 months from the date of joining the Company. This training period may be extended based on your performance and at the discretion of the Company. Post the successful completion of this training period, you will be placed on Probation. You will be entitled to a monthly stipend of ₹ 20,000/- during the training period.

2. Probationary Period:

You will be on probation for a period of 3 months from the date of completion of your training period and may be confirmed as a permanent employee upon successful completion of your probation. If the Company does not find your performance satisfactory, it reserves the right to not proceed with employment.

3. Background Check:

Your confirmation of employment is subject to you submitting the requisite documents as required by the Company and successful completion of background checks to validate your identity. You undertake to provide authentic, true, and complete information and provide your consent to use the information for the purpose of conducting the background check. If the outcome of the background check is not desirable, the Company reserves the right to withdraw this Offer without notice and Compensation and/or termination of your employment.

4. Notice Period:

You will be required to give a 3 month notice or salary thereof in case you decide to leave our services subject to the Company's discretion. In the event of you having any incomplete assignment, the Company will have the discretion to relieve you only at the end of the three-month notice period. Similarly, the Company can terminate your services by giving 3 months' notice or fixed salary thereof. For reasons of clarity, during your probation period, you are required to give a notice of 3 months. Similarly, the Company can terminate your services by giving you a notice of 3 months, during your probation period.

5. Confidential Information:

You will not divulge or communicate to any other person or persons, natural or juristic, directly or indirectly, any information about the affairs of the Company or the management at any time or any other information deemed confidential to the Company. This includes but not limited to any Intellectual Property (IP) of the Company, or any IP developed by you, during your term at the Company.

6. Other Terms & Conditions:

You agree not to undertake employment, whether full-time or part-time, as the Director / Partner / Member / Employee of any other organization / entity engaged in any form of business activity without the consent of the Company and in accordance with Company policies.

ACKNOWLEDGEMENTS

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide **Mr. H. Senthil Kumar**, Assistant Professor, Presidency School of Computer Science and Engineering, Presidency University, for his moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to **Dr. Asif Mohammed**, Head of the Department, Presidency School of Computer Science and Engineering Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

I am grateful to **Dr. Md Ziaur Rahaman,** Internship Coordinator and **Mr. Sakthi S,** Program Internship Coordinator, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

Chirag BA

ABSTRACT

The rapid evolution of data analytics and artificial intelligence has significantly transformed how complex problems are approached across various domains, including sports analytics, business intelligence, and natural language processing. This internship focuses on the practical application of statistical modeling, simulation techniques, structured database querying, and large language model (LLM) evaluation frameworks to solve real-world problems. The work carried out during this internship integrates multiple interdisciplinary concepts, enabling a comprehensive understanding of data-driven decision-making systems.

A key component of the internship involved the development of a Monte Carlo Simulation model to predict cricket match outcomes. The simulation model operates by generating thousands of ball-by ball match scenarios based on probabilistic distributions derived from historical player performance data. As observed in the simulation process illustrated in the uploaded material, batting probabilities are calculated based on the likelihood of scoring runs such as 0, 1, 4, or 6, while bowling probabilities incorporate wicket-taking ability and risk factors . By repeatedly simulating matches, the system estimates the probability of winning for each team, thereby providing a statistically grounded prediction mechanism.

Another significant project undertaken during the internship was the development of an IPL fantasy team selection system using a data-driven approach. This system utilizes multiple player performance metrics, including strike rate, total runs, boundaries, wickets taken, and economy rate, which are normalized into a standard scale to enable fair comparison across players. The normalized metrics are then combined using a weighted scoring model to compute an overall impact score. As described in the project, optimization techniques such as linear programming and knapsack constraints were applied to construct an optimal team within budget limitations and predefined rules, ensuring maximum performance efficiency .

In addition to simulation and analytics, the internship involved extensive work in advanced SQL based data analysis. Complex queries were developed using recursive common table expressions (CTEs), window functions, and aggregation techniques to analyze hierarchical data, calculate running totals, and perform segmentation tasks. These techniques enabled efficient extraction of insights from structured datasets and demonstrated the importance of optimized query design in

handling large-scale data .

A particularly advanced aspect of the internship was the evaluation and improvement of large language model outputs. This task involved designing prompts for various use cases, defining evaluation rubrics, and systematically scoring responses generated by different models such as Gemini, ChatGPT, and Claude. Each response was evaluated based on both writing quality and non writing quality criteria, with scores assigned accordingly. The responses were then ranked, and the best-performing output was refined to generate a “golden response” that satisfies all evaluation criteria. This process provided valuable insights into prompt engineering, response evaluation, and iterative improvement of AI-generated content.

Overall, the internship provided a comprehensive learning experience by integrating concepts from data science, simulation modeling, database systems, and artificial intelligence. The knowledge gained through these activities has significantly enhanced both technical expertise and problem solving capabilities.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
	Declaration	iii
	Acknowledgement	v
	Abstract	vi
	List of Figures	ix
	List of Tables	x
	Abbreviations	xi
1.	Introduction	
	1.1 Background	
	1.2 Statistics of Internship	
	1.3 Prior existing technologies	
	1.4 Proposed approach	
	1.5 Objectives	
	1.6 SDGs	
	1.7 Overview of internship report	
2.	Literature Review	
3.	Methodology	
4.	Project Management	
	4.1 Internship timeline	
	4.2 Risk analysis	
	4.3 Internship budget	
5.	Analysis and Design	
	5.1 Requirements	
	5.2 Block Diagram	
	5.3 System Flow Chart	
	5.4 Choosing devices	
	5.5 Designing units	
	5.6 Standards	
	5.7 Mapping with IoTWF reference model layers	
	5.8 Domain model specification	
	5.9 Communication model	
	5.10 IoT deployment level	

5.11 Functional view
5.12 Mapping IoT deployment level with functional view
5.13 Operational view
5.14 Other Design
6. Hardware, Software and Simulation
6.1 Hardware
6.2 Software development tools
6.3 Software code
6.4 Simulation
7. Evaluation and Results
7.1 Test points
7.2 Test plan
7.3 Test result
7.4 Insights
8. Social, Legal, Ethical, Sustainability and Safety Aspects
8.1 Social aspects
8.2 Legal aspects
8.3 Ethical aspects
8.4 Sustainability aspects
8.5 Safety aspects
9. Conclusion
References
Base Paper (Optional)
Appendix

LIST OF FIGURES

Sl. No.	Figure Name	Caption	Page No.
1	Figure 3.1	Batting Probability and Cumulative Probability	08
2	Figure 3.2	Bowling Probability and Cumulative Probability	08
3	Figure 3.3	Impact Score Formula	09
4	Figure 3.4	LLM Evaluation Rubrics	11
5	Figure 3.5	Golden Response Generation	11
6	Figure 5.1	Internship architecture	17
7	FIgure 5.2	Monte Carlo Simulation Process	19
8	FIgure 5.3	IPL Team Optimisation	20
9	FIgure 5.4	Python Data Processing Workflow	21
10	FIgure 5.5	LLM Evaluation Architecture	23
11	FIgure 7.1	Monte Carlo KPI Cards	31
12	FIgure 7.2	IPL Full Squad	32
13	FIgure 7.3	LLM Golden Response	34

LIST OF TABLES

Sl. No.	Table Name	Table Caption	Page No.
1	Table 7.1	System Comparison	34

ABBREVIATIONS

Abbreviation	Full Form
IPL	Indian Premier League
SQL	Structured Query Language
AI	Artificial Intelligence
CTE	Common Table Expression
ML	Machine Learning

Chapter 1

INTRODUCTION

1.1 Background

In the modern era of technology, data has become one of the most valuable resources for decision-making across industries. Organizations increasingly rely on data analytics and artificial intelligence to extract meaningful insights, optimize processes, and improve overall performance. In domains such as sports analytics, the use of statistical models and simulation techniques has enabled teams and analysts to move beyond intuition-based decisions toward more objective, data-driven strategies. Similarly, in business environments, structured data analysis using database systems allows organizations to understand customer behavior, track performance metrics, and identify growth opportunities.

At the same time, the emergence of large language models (LLMs) has revolutionized the way machines interact with human language. These models are capable of generating coherent and contextually relevant responses across a wide range of applications, including content generation, summarization, and conversational systems. However, despite their capabilities, evaluating the quality and reliability of LLM-generated responses remains a critical challenge. This has led to the development of structured evaluation frameworks that use predefined rubrics to assess model performance objectively.

This internship was designed to provide hands-on exposure to these evolving technologies by combining multiple domains such as simulation modeling, data analytics, database management, and artificial intelligence evaluation. By working on real-world inspired problems, including cricket match prediction, fantasy team optimization, and AI response evaluation, the internship offers a holistic understanding of how theoretical concepts can be applied to practical scenarios.

1.2 Statistics of Internship

The internship encompassed a diverse range of projects that collectively contributed to the development of analytical, technical, and problem-solving skills. The work carried out can be broadly categorized into four major domains: simulation modeling, data analytics,

database systems, and artificial intelligence evaluation. Each of these domains involved the application of advanced techniques and tools, providing a well-rounded learning experience.

In the domain of simulation modeling, a Monte Carlo-based system was developed to predict cricket match outcomes by simulating thousands of possible scenarios. This involved the use of probabilistic distributions derived from historical data to model real-world uncertainty. In the field of data analytics, a comprehensive IPL fantasy team selection system was designed, where player performance metrics were analyzed and optimized using mathematical models and constraints. The database component of the internship involved writing advanced SQL queries to perform complex data analysis tasks such as hierarchical traversal, ranking, and aggregation. Finally, the artificial intelligence component focused on evaluating and improving responses generated by large language models using structured rubrics and scoring mechanisms.

Through these activities, the internship provided exposure to a wide range of technologies and methodologies, enabling the development of both technical expertise and analytical thinking.

1.3 Prior Existing Technologies

Before the implementation of the approaches developed during this internship, several traditional methods were commonly used in the respective domains. In sports analytics, predictions were often based on basic statistical measures such as averages, historical win rates, or expert opinions. While these methods provided some level of insight, they were limited in their ability to account for uncertainty and variability inherent in real-world scenarios. Simulation techniques such as Monte Carlo methods offer a more advanced alternative by incorporating randomness and generating probabilistic outcomes, thereby providing a more realistic representation of match dynamics.

In the context of fantasy team selection, conventional approaches relied heavily on subjective judgment or reputation-based selection of players. However, with the availability of large datasets and advanced analytical tools, data-driven methods have become increasingly popular. These methods enable objective evaluation of players based on performance metrics, ensuring more accurate and optimized team selection.

In database systems, basic SQL queries were traditionally used for data retrieval and simple analysis. However, modern data analysis requires more sophisticated techniques such as

recursive queries, window functions, and complex aggregations. These advanced techniques allow for deeper insights and more efficient handling of large datasets.
Similarly, in the field of artificial intelligence, the evaluation of model outputs was often subjective and inconsistent. The introduction of rubric-based evaluation frameworks provides a structured approach to assessing the quality of responses, ensuring consistency and enabling systematic improvement of model performance.

1.4 Proposed Approach

The approach adopted during this internship integrates advanced techniques from multiple domains to overcome the limitations of traditional methods. For cricket match prediction, a Monte Carlo simulation model was implemented, where each ball in a match is simulated based on probability distributions derived from historical data. As illustrated in the simulation workflow (refer to Figure 1), the model generates random outcomes for each ball and updates the match state accordingly. By repeating this process thousands of times, the system estimates the probability of each team winning, providing a robust and statistically sound prediction mechanism .

For the IPL fantasy team selection project, a data-driven approach was adopted to evaluate player performance. Multiple metrics were collected and normalized to ensure comparability across different scales. These metrics were then combined using a weighted scoring model to calculate an overall impact score for each player. Optimization techniques such as linear programming and knapsack constraints were applied to select the best possible team within budget and rule constraints.

In the database component, advanced SQL techniques were utilized to analyze structured data efficiently. Recursive queries were used to explore hierarchical relationships, while window functions enabled ranking and cumulative calculations. These techniques allowed for the extraction of meaningful insights from complex datasets.
For the LLM evaluation project, a systematic framework was developed to assess and improve model-generated responses.

Chapter 2

LITERATURE SURVEY

2.1 Introduction to Literature Review

A literature review plays a crucial role in understanding the existing research, methodologies, and technologies related to a particular problem domain. It provides a foundation for identifying gaps in current approaches and justifies the need for improved or alternative solutions. In the conteAPIxt of this internship, the literature review focuses on four primary domains: simulation-based modeling, sports analytics and optimization, database systems and SQL analytics, and large language model (LLM) evaluation frameworks.

These domains have evolved significantly over the years, driven by advancements in computational power, availability of large datasets, and the development of sophisticated algorithms. By analyzing existing work in these areas, it becomes possible to understand how traditional methods operate, what limitations they possess, and how modern approaches improve upon them. This chapter reviews relevant concepts and techniques that form the theoretical basis for the work carried out during the internship.

2.2 Simulation Techniques and Monte Carlo Methods

Simulation techniques have long been used to model complex systems where analytical solutions are either difficult or impossible to derive. Among these techniques, the Monte Carlo simulation method is one of the most widely used approaches for handling uncertainty and randomness in systems. It involves generating random samples from probability distributions and using these samples to approximate the behavior of a system over time.

Monte Carlo methods are particularly effective in scenarios where outcomes depend on multiple uncertain variables. In sports analytics, for example, match outcomes depend on numerous factors such as player performance, pitch conditions, and game dynamics. Traditional statistical methods often fail to capture this complexity, whereas simulation-based approaches provide a more realistic representation by modeling each event probabilistically.

As observed in the simulation design used during the internship, the Monte Carlo model operates by simulating cricket matches on a ball-by-ball basis. Each ball outcome is determined using probability distributions derived from historical batting and bowling data. For instance, the probability of scoring different runs or taking a wicket is calculated from past records and used to generate random outcomes. This process is repeated thousands of times to estimate the likelihood of different match results .

The strength of Monte Carlo simulation lies in its ability to model real-world uncertainty and provide probabilistic insights rather than deterministic predictions. However, it also has limitations, including computational complexity and dependence on the accuracy of input data. Despite these challenges, it remains a powerful tool for predictive modeling in various domains.

2.3 Data Analytics in Sports and Team Optimization

The application of data analytics in sports has gained significant traction in recent years, transforming how teams evaluate players, design strategies, and make decisions. In cricket, performance analysis involves examining multiple metrics such as batting averages, strike rates, boundary percentages, wickets taken, and economy rates. These metrics provide insights into a player's strengths and weaknesses, enabling more informed decision-making.

One of the key challenges in sports analytics is the presence of multiple performance metrics that exist on different scales. For example, runs scored may range in hundreds, while strike rates are typically between 80 and 200. Direct comparison of such metrics can lead to biased results. To address this issue, normalization techniques are used to transform all metrics into a common scale, usually between 0 and 1. This ensures that each metric contributes proportionally to the overall evaluation.

In the IPL fantasy team selection project developed during the internship, a weighted scoring model was used to calculate an impact score for each player. Different weights were assigned to various metrics based on their importance in the T20 format. For instance, strike rate and boundary percentage were given higher importance due to their direct impact on scoring efficiency, while other metrics provided supporting context. The normalized and weighted metrics were combined to generate a comprehensive performance score for each player .

Furthermore, optimization techniques such as linear programming and knapsack algorithms

were used to select the best possible team within given constraints. These constraints included budget limits, squad size, and team composition rules. The use of such optimization models ensures that the selected team maximizes overall performance while adhering to practical limitations. This approach represents a significant improvement over traditional methods that rely on subjective judgment.

2.4 Database Systems and Advanced SQL Techniques

Database systems play a fundamental role in storing, managing, and analyzing large volumes of structured data. SQL (Structured Query Language) is widely used for interacting with relational databases and performing data analysis tasks. While basic SQL queries are sufficient for simple data retrieval, advanced analytical tasks require more sophisticated techniques.

One such technique is the use of recursive common table expressions (CTEs), which allow for the traversal of hierarchical data structures. For example, in organizational databases, employees may be linked through reporting relationships, forming a hierarchy. Recursive queries enable the analysis of such structures by iteratively exploring parent-child relationships. As demonstrated in the SQL work performed during the internship, recursive CTEs were used to calculate the number of employees under each manager and determine the depth of organizational hierarchies .

Another important concept in advanced SQL is the use of window functions. These functions allow calculations to be performed across a set of rows related to the current row, without collapsing the result into a single aggregated value. For instance, functions such as ROW_NUMBER(), RANK(), and SUM() OVER() are used to perform ranking, cumulative calculations, and segmentation. These techniques are essential for tasks such as identifying top-performing entities, calculating running totals, and analyzing trends over time.

Additionally, complex analytical queries often involve joins, subqueries, and aggregation functions to combine and process data from multiple tables. As seen in the SQL assignments completed during the internship, these techniques were used to analyze customer behavior, sales performance, and user activity patterns .

Overall, advanced SQL techniques provide powerful tools for extracting meaningful insights from structured data, making them an essential component of modern data analytics.

2.5 Large Language Models and Prompt Engineering

Large Language Models (LLMs) such as Gemini, ChatGPT, and Claude represent a significant advancement in artificial intelligence, particularly in the field of natural language processing. These models are trained on vast amounts of textual data and are capable of generating human-like responses across a wide range of tasks. However, the quality of their output depends heavily on the design of the input prompt, making prompt engineering a critical aspect of their effective use.

Prompt engineering involves carefully crafting input instructions to guide the model toward producing accurate, relevant, and high-quality responses. Different prompts can lead to significantly different outputs, even when the underlying model remains the same. This highlights the importance of understanding how to structure prompts effectively.

In addition to prompt design, evaluating the quality of LLM outputs is a key challenge. Traditional evaluation methods are often subjective and inconsistent. To address this issue, structured evaluation frameworks have been developed, where responses are assessed using predefined rubrics. These rubrics can be categorized into writing quality metrics, such as clarity and coherence, and non-writing quality metrics, such as factual correctness and completeness.

During the internship, a systematic evaluation process was implemented in which responses generated by different models were scored based on these rubrics. Writing quality metrics were assigned scores between 0 and 1, while non-writing quality metrics were evaluated using binary values (true or false). Based on these scores, responses were ranked, and the best-performing response was further refined to create a “golden response” that satisfies all evaluation criteria.

This approach ensures a structured and objective evaluation of LLM outputs and highlights the importance of iterative improvement in achieving high-quality results.

2.6 Critical Analysis of Existing Approaches

While traditional analytical methods have been widely used across domains such as sports analytics, database systems, and predictive modeling, they often exhibit significant limitations when applied to complex real-world problems. Conventional statistical

techniques primarily rely on aggregated metrics such as averages, totals, and historical summaries. Although these methods provide a basic understanding of patterns, they fail to capture the dynamic and uncertain nature of real-world scenarios. For instance, in sports analytics, match outcomes are influenced by multiple unpredictable factors, including player form, pitch conditions, and match pressure, which cannot be fully represented through static statistical measures.

Simulation-based approaches such as Monte Carlo methods address these limitations by introducing probabilistic modeling and randomness into the analysis. These methods generate multiple possible outcomes and provide a distribution of results rather than a single deterministic prediction. However, their effectiveness depends heavily on the quality and accuracy of input data, as well as computational resources required for large-scale simulations. Similarly, data-driven approaches in analytics rely on the availability of structured and reliable datasets. Any inconsistencies or missing values in the data can significantly impact the accuracy of the results.

In the domain of database systems, traditional SQL queries are effective for retrieving and filtering data but are limited in their ability to perform advanced analytical operations. The introduction of techniques such as window functions and recursive queries has significantly enhanced the analytical capabilities of SQL, enabling deeper insights into complex datasets. In the field of artificial intelligence, particularly in evaluating large language models, subjective judgment has historically been a major challenge. The use of structured evaluation rubrics provides a more consistent and objective framework for assessing model performance, addressing the limitations of purely qualitative evaluation methods.

2.7 Summary of Literature Review

The literature review highlights the importance of advanced analytical techniques in addressing complex real-world problems. Simulation methods such as Monte Carlo provide a powerful approach for modeling uncertainty and predicting outcomes. Data analytics techniques enable objective evaluation and optimization in domains such as sports. Advanced SQL methods facilitate efficient analysis of large datasets, while prompt engineering and evaluation frameworks play a crucial role in improving the performance of

large language models.

The concepts and methodologies discussed in this chapter form the theoretical foundation for the work carried out during the internship. By integrating these techniques, the internship demonstrates how multiple domains can be combined to develop comprehensive and effective solutions

Chapter 3

Methodology

3.1 Introduction

The methodology adopted during this internship focuses on the systematic application of analytical, computational, and evaluation techniques to solve real-world problems across multiple domains. The work carried out integrates simulation modeling, data analytics, database querying, and artificial intelligence evaluation into a cohesive framework. Each component of the internship follows a structured process involving data collection, preprocessing, model development, execution, and evaluation.

The methodology is divided into four major sections based on the projects undertaken: Monte Carlo simulation for match prediction, IPL fantasy team optimization, advanced SQL-based data analysis, and large language model (LLM) evaluation. Each of these components follows a logical workflow designed to ensure accuracy, efficiency, and scalability.

3.2 Monte Carlo Simulation Methodology

The Monte Carlo simulation model was developed to predict the outcomes of cricket matches by modeling the inherent uncertainty present in the game. The methodology begins with the collection of historical batting and bowling data, which forms the foundation for probability calculations. This data includes metrics such as runs scored per ball, frequency of boundaries, wicket-taking rates, and extras conceded.

The first step in the simulation process involves transforming raw historical data into probability distributions. For batting, the probability of scoring different outcomes (such as 0, 1, 2, 4, or 6 runs) is calculated by dividing the frequency of each outcome by the total number of balls faced. These probabilities are then converted into cumulative probability ranges, which are used to map random numbers to specific outcomes. As illustrated in the simulation design, cumulative probability enables the creation of a continuous range between 0 and 1, allowing random values to determine match events .

Dot Probability	1's Probability	2's Probability	3's Probability	4's Probability	6's Probability	CP(Dots)	CP(1)	CP(2)	CP(3)	CP(4)	CP(6)
0.332	0.417	0.076	0.004	0.130	0.042	0.332	0.749	0.825	0.829	0.958	1.000
0.393	0.351	0.049	0.000	0.098	0.108	0.393	0.744	0.793	0.793	0.892	1.000
0.381	0.268	0.058	0.000	0.194	0.100	0.381	0.648	0.706	0.706	0.900	1.000
0.380	0.345	0.063	0.002	0.117	0.095	0.380	0.724	0.787	0.788	0.905	1.000
0.363	0.380	0.060	0.001	0.128	0.068	0.363	0.743	0.803	0.804	0.932	1.000
0.372	0.385	0.078	0.000	0.112	0.054	0.372	0.757	0.834	0.834	0.946	1.000
0.657	0.229	0.057	0.000	0.057	0.000	0.657	0.886	0.943	0.943	1.000	1.000
0.667	0.333	0.000	0.000	0.000	0.000	0.667	1.000	1.000	1.000	1.000	1.000
0.625	0.375	0.000	0.000	0.000	0.000	0.625	1.000	1.000	1.000	1.000	1.000
0.417	0.333	0.125	0.000	0.083	0.042	0.417	0.750	0.875	0.875	0.958	1.000
0.647	0.324	0.000	0.000	0.000	0.029	0.647	0.971	0.971	0.971	0.971	1.000

Fig 3.1 - Batting Probablity and Cumulative Probablity

Similarly, bowling performance is modeled using probabilities of taking wickets and conceding extras. These probabilities are derived from historical bowling data and incorporated into the simulation logic. The combination of batting and bowling probabilities ensures that the simulation captures both offensive and defensive aspects of the game.

The core of the simulation lies in generating random numbers to simulate each ball. For every delivery, a random value between 0 and 1 is generated and compared against the cumulative probability distribution of the batsman. Based on this comparison, the outcome of the ball is determined. Additional checks are performed to account for wickets and extras, ensuring a realistic representation of match dynamics.

The match is simulated ball-by-ball for a fixed number of overs or until all wickets are lost. This process is repeated thousands of times to generate a large sample of possible outcomes. The final result is obtained by calculating the proportion of simulations in which each team wins. This provides a probabilistic estimate of match outcomes, offering deeper insights compared to traditional deterministic methods.

Probabilities				Bowling Cummulative Probabilities			
Wide	No balls	Wicket	Good ball	Wide	No balls	Wicket	Good ball
0.037	0.005	0.045	0.913	0.037	0.042	0.087	1.000
0.034	0.005	0.058	0.903	0.034	0.039	0.097	1.000
0.000	0.000	0.045	0.955	0.000	0.000	0.045	1.000
0.042	0.011	0.047	0.900	0.042	0.053	0.100	1.000
0.067	0.007	0.049	0.877	0.067	0.074	0.123	1.000
0.021	0.002	0.040	0.937	0.021	0.023	0.063	1.000

Fig 3.2 - Bowling Probablity and Cumulative Probablity

3.3 IPL Fantasy Team Selection Methodology

The IPL fantasy team selection project follows a data-driven methodology aimed at optimizing team composition based on player performance metrics. The process begins with the collection of player statistics from reliable sources, including official IPL data and

cricket analytics platforms. The collected data includes metrics such as total runs, strike rate, number of boundaries, batting average, wickets taken, economy rate, and matches played.

Once the data is collected, it undergoes a preprocessing phase to ensure accuracy and consistency. This includes removing invalid records, handling missing values, and converting data into appropriate numerical formats. As identified during the project, certain anomalies such as zero economy rates were corrected by replacing them with average values to prevent distortion of performance scores .

A key step in the methodology is normalization, which converts all performance metrics into a common scale. This is necessary because different metrics have different ranges and units. By normalizing the data, each metric contributes proportionally to the overall evaluation, ensuring fairness in comparison.

The next step involves calculating the impact score for each player using a weighted scoring model. For batting performance, metrics such as strike rate, total runs, and boundary percentage are assigned higher weights due to their importance in T20 cricket. For bowling performance, wickets are given primary importance, while economy rate is treated as a penalty factor. All-rounders are evaluated using a combination of batting and bowling scores.

- Impact_Score :
  (Batting_Output_Score + Bowling_Output_Score) / 2
- Impact_Score:
  (Runs + Strike_Rate + Batting_Avg) / 3

Fig 3.3 - Impact Score Formula

After calculating individual player scores, an optimization model is applied to select the best possible team. This model considers constraints such as budget limits, squad size, role distribution, and overseas player restrictions. Techniques such as linear programming and the knapsack algorithm are used to maximize the total impact score while satisfying all constraints.

The final output is an optimized team that balances performance, cost, and strategic requirements, demonstrating the effectiveness of data-driven decision-making.

3.4 Advanced SQL Analysis Methodology

The SQL component of the internship focuses on extracting insights from structured datasets using advanced querying techniques. The methodology begins with understanding the database schema, including relationships between tables such as customers, orders, products,

and employees. This understanding is essential for constructing efficient and meaningful queries.

Basic queries are used initially to retrieve and filter data, followed by more complex queries that involve joins, aggregations, and subqueries. These queries enable the analysis of various aspects such as customer behavior, sales trends, and product performance.

A significant part of the methodology involves the use of recursive common table expressions (CTEs) to analyze hierarchical data. For example, recursive queries are used to determine the number of employees under each manager and to calculate the depth of organizational structures. These queries iteratively process data, allowing for the exploration of complex relationships within the dataset .

Window functions are another key component of the methodology. These functions allow calculations to be performed across a set of rows related to the current row. For instance, ranking functions such as ROW_NUMBER() and RANK() are used to identify top-performing entities, while cumulative functions such as SUM() OVER() are used to calculate running totals.Additionally, advanced analytical queries are used to perform segmentation, correlation analysis, and performance evaluation. For example, queries are designed to identify high-value customers, analyze sales distribution across regions, and evaluate conversion rates. These techniques demonstrate the power of SQL in handling complex analytical tasks efficiently .

3.5 LLM Evaluation and Prompt Engineering Methodology

The LLM evaluation project follows a structured methodology aimed at assessing and improving the quality of responses generated by large language models. The process begins with defining specific use cases and designing prompts that clearly communicate the desired output. The effectiveness of a prompt plays a critical role in determining the quality of the generated response.

Once the prompts are defined, evaluation rubrics are created to assess the responses. These rubrics are categorized into two types: writing quality and non-writing quality. Writing quality metrics include aspects such as clarity, coherence, structure, and fluency, while non-writing quality metrics focus on factual correctness, completeness, and adherence to instructions.

Each rubric is further classified as either critical or non-critical, depending on its importance in the evaluation process. Responses generated by different models, such as Gemini,

ChatGPT, and Claude, are then evaluated against these rubrics. Writing quality metrics are scored on a scale from 0 to 1, while non-writing quality metrics are evaluated using binary values (true or false).

Rubric Type	Property/ Rubrics	Rubric criticality	Gemini 3 Pro Response	Is property True/False for gemini-3-pro-preview Response	Justification if FALSE	ChatGPT 5.2 Response	Is property True/False for ChatGPT 5.2 Response	Justification if FALSE	Claude-opus 4.6 Response	Is property True/False for Claude-opus 4.6 Response	Justification if FALSE
Generic Rubrics	Clarity & Readability: Measure	Critical	We are currently	TRUE		**Equities:** Van	TRUE		| U.S. fiscal crisis	TRUE	
Generic Rubrics	Accuracy & Financial Correctn	Critical	* **U.S. Equitie	TRUE		**International ec	TRUE		| AI-driven produ	TRUE	
Generic Rubrics	Completeness / Coverage: As	Critical	* **Internationa	TRUE			TRUE		---	TRUE	
Generic Rubrics	Structure & Organization: Eval	Critical	* **Fixed Incom	TRUE		**Emerging mark	TRUE			TRUE	
Generic Rubrics	Consistency: Checks whether	Critical	### 3. Portfolio F	TRUE		**Fixed income:*	TRUE		## VI. Conclusio	TRUE	
Generic Rubrics	Data References & Evidence:	Critical	Given elevated i	FALSE	The response in		FALSE	While the respor	The investment e	TRUE	
Generic Rubrics	Audience Appropriateness: As	Critical		TRUE		## Portfolio posit	TRUE			TRUE	
Generic Rubrics	Analytical Clarity: Measures wl	Critical	**Asset Allocatio	TRUE		Vanguard's valua	TRUE		**Our core convi	TRUE	
Generic Rubrics	Compliance & Professional Sta	Critical	* **Underweigh	FALSE	The response or		FALSE	The response on	1. **Fixed incom	TRUE	
Generic Rubrics	Language Quality: Grammar, s	Critical	* **Equity Sub-	TRUE		- **Equity positio	TRUE			TRUE	
Generic Rubrics	Tone and Voice: Tone should b	Critical	* **Fixed Incom	FALSE	The tone occasio	- **Fixed income	FALSE	Although genera	2. **U.S. equity v	FALSE	Although signific
Generic Rubrics	Style Guide Alignment: Adhere	Non-critical	*Expected Outco	FALSE	The response do	## Key factors in	FALSE	The response pa	3. **Diversificatio	TRUE	
Generic Rubrics	Information Density: Does the	Critical	### 4. Key Factc	TRUE			TRUE			TRUE	
Generic Rubrics	Nonjudgemental Tone: Does th	Non-critical	Moving through	TRUE		1. **Supply-side	TRUE		4. **The era of su	TRUE	
Generic Rubrics	Writing Naturalness: Does the	Non-critical	1. **Policy and	FALSE	The writing, whil	2. **Policy path a	FALSE	The response m	We will continue	FALSE	While the respo
Use-case Specif	Market-to-Portfolio Narrative L	Critical	2. **Productivity	TRUE		3. **Valuation co	TRUE			TRUE	
Use-case Specif	Cautious Market Commentary	Critical	3. **Tug-of-War	FALSE	The response pr	4. **Rates regim	FALSE	The response pr	---	FALSE	The response in
Use-case Specif	Sectioned Commentary Rhythi	Critical	**Conclusion:**	TRUE		5. **Trade/geopo	TRUE		*This report is ba	TRUE	
			We remain confi			Overall, Vanguar					

Fig 3.4 - LLM Evaluation Rubrics

After scoring, the responses are ranked based on their overall performance. The highest-ranked response is then refined to create a "golden response," which represents the ideal output. This refinement process involves improving clarity, correcting errors, and ensuring that all rubric criteria are satisfied.

Model Ranking						Model Ranking pairwise Justification			Golden_Response The perfect, masterful response. You may create the golden response(perfect response) by revising the Gemini response or creating golden response from scratch. DO NOT use or reference non-gemini model response to craft golden response The perfect response should pass all rubrics (both writing quality rubrics and non-writing-quality rubrics).
Gemini 3 Pro	Justification	ChatGPT 5.2	Justification	Claude-opus 4.6	Justification	gemini 3 vs gpt5.2	gpt5.2 vs opus4.6	gemini 3 vs opus4.6	
									Key benefit: Enhanced income generation and downside protection
									This environment restores bonds' dual role as both return generators and portfolio stab
									3. Portfolio Positioning: Valuation-Aware Allocation (TVAA)
									Given elevated equity valuations and a compressed equity risk premium, the portfolio is
									Strategic Allocation
2	Response is a st	2	Response is clea	1		Both Gemini 3 P	Claude-opus 4.6	Claude-opus 4.6	Equities: 38% (Underweight)
									Fixed Income: 62% (Overweight)
									Equity Positioning
									Overweight:
									U.S. Value (13%)
									U.S. Small-Cap (5%)
									Developed ex-U.S. (11%)

Fig 3.5 - Golden Response Generation

This methodology ensures a systematic and objective approach to evaluating LLM outputs and highlights the importance of iterative improvement in achieving high-quality results.

3.7 Data Preprocessing and Its Importance

Data preprocessing is a critical step in any analytical workflow, as the quality of input data directly determines the reliability and accuracy of the output. In real-world scenarios, raw data is often incomplete, inconsistent, or contains errors that can negatively impact the results of analysis. Therefore, preprocessing involves a series of steps, including data cleaning, transformation, normalization, and validation, to ensure that the data is suitable for further processing.

In the context of simulation modeling, preprocessing involves calculating probability distributions from historical data. This requires accurate aggregation of past events and careful handling of anomalies. In the IPL analytics system, preprocessing includes normalizing player performance metrics to ensure that different attributes, such as runs and strike rates, can be compared on a common scale. Without normalization, metrics with larger numerical ranges may dominate the analysis, leading to biased results.

In SQL-based systems, preprocessing may involve filtering irrelevant records, restructuring data, and ensuring referential integrity. This step is essential for improving query performance and ensuring meaningful outputs. Overall, effective preprocessing enhances the quality of analysis, reduces computational errors, and ensures that the results are both accurate and reliable.

3.8 Python-Based Data Processing Methodology

The Python notebook developed during the internship focuses on solving real-world data processing problems using structured programming techniques and efficient data handling methods. The methodology involves processing nested and semi-structured datasets, performing iterative traversal, and applying conditional logic to extract meaningful insights.

The approach begins with representing data in the form of dictionaries and lists, which allows for flexible storage of hierarchical information. Each dataset is processed using nested loops, where the outer loop iterates through primary entities such as customers or categories, and inner loops traverse sub-level data such as transactions or attributes. Conditional statements are used to filter relevant information, ensuring that only required data is processed.

Aggregation is performed by maintaining a separate data structure, typically a dictionary, where computed values such as totals or counts are stored and updated dynamically. This approach ensures efficient computation and avoids redundant processing. The methodology emphasizes clarity, modularity, and correctness, making it suitable for handling complex data transformation tasks.

Chapter 4

Project Management

4.1 Introduction

Project management is a critical component in ensuring the successful execution of any technical project. It involves planning, organizing, monitoring, and controlling various activities to achieve predefined objectives within specified constraints such as time, resources, and scope. During this internship, multiple projects were carried out simultaneously, including simulation modeling, data analytics, SQL-based analysis, and large language model evaluation. Managing these diverse tasks required a structured approach to ensure timely completion and efficient utilization of resources.

The project management methodology adopted during this internship focused on dividing the overall work into smaller, manageable phases. Each phase was carefully planned with defined objectives, timelines, and expected outcomes. This approach ensured that progress could be monitored effectively and any issues could be addressed in a timely manner.

4.2 Project Planning

The planning phase involved identifying the scope of the internship and defining the key objectives of each project. The work was divided into four major components: Monte Carlo simulation, IPL fantasy team optimization, SQL data analysis, and LLM evaluation. Each component required a different set of skills and tools, making it essential to allocate time and effort appropriately.

A step-by-step plan was created to guide the execution of each project. The simulation project began with data collection and probability modeling, followed by implementation and testing. The IPL analytics project involved data gathering, preprocessing, scoring model development, and optimization. The SQL component required understanding database schemas and writing progressively complex queries. The LLM evaluation project involved prompt design, rubric creation, response scoring, and refinement.

This structured planning ensured that all tasks were aligned with the overall objectives of the internship and helped maintain a clear direction throughout the project lifecycle.

4.3 Project Timeline

The internship activities were carried out over a defined period, with each project allocated a specific time frame. The work was organized into sequential and overlapping phases to ensure efficient use of time.

The initial phase focused on understanding the problem statements and collecting relevant data. This was followed by the development phase, where models and queries were implemented. The testing and evaluation phase involved validating the results and refining the outputs. Finally, the documentation phase ensured that all work was properly recorded and presented.

The timeline was designed to allow flexibility, enabling adjustments based on the complexity of tasks and any challenges encountered during implementation. This adaptive approach ensured that the quality of work was not compromised while meeting deadlines.

4.4 Resource Management

Effective resource management is essential for the successful execution of any project. In this internship, resources primarily included software tools, datasets, and computational capabilities. Tools such as Google Sheets were used for data collection and preprocessing, while SQL environments were used for database analysis. Additionally, programming environments were utilized for implementing simulation logic and evaluating results.

The availability and proper utilization of datasets played a crucial role in the success of the projects. Reliable data sources such as cricket statistics websites were used to ensure the accuracy of simulation and analytics models. Care was taken to validate and preprocess the data to avoid errors and inconsistencies.

Time was also treated as a critical resource, and efforts were made to allocate sufficient time for each phase of the project. By managing resources effectively, the internship was able to achieve its objectives within the given constraints.

4.5 Risk Management

Risk management involves identifying potential challenges and developing strategies to mitigate their impact. During the internship, several risks were identified that could affect

the quality and progress of the projects.

One of the primary risks was related to data quality. Incomplete or inaccurate data could lead to incorrect results in both simulation and analytics models. To address this issue, data preprocessing techniques were applied, including handling missing values and correcting anomalies such as unrealistic metrics.

Another risk involved the complexity of algorithms and queries. Advanced techniques such as Monte Carlo simulation and recursive SQL queries require careful implementation to avoid errors. This risk was mitigated by testing the models thoroughly and validating the results at each stage.

Time constraints also posed a potential risk, especially when working on multiple projects simultaneously. This was managed by prioritizing tasks and following a structured timeline to ensure timely completion.

By identifying and addressing these risks proactively, the internship was able to maintain a high standard of work and minimize potential disruptions.

4.6 Quality Assurance

Ensuring the quality of outputs was a key focus throughout the internship. Various techniques were employed to validate the accuracy and reliability of the results. In the simulation project, multiple iterations were performed to ensure consistent probability estimates. In the analytics project, the impact scoring model was tested with different datasets to verify its effectiveness.

For SQL queries, results were cross-verified to ensure correctness, especially for complex queries involving multiple joins and aggregations. In the LLM evaluation project, responses were carefully reviewed against predefined rubrics to ensure that the scoring process was consistent and objective.

This emphasis on quality assurance ensured that the final outputs were accurate, reliable, and aligned with the objectives of the internship.

Chapter 5

Analysis and Design

5.1 Introduction

The analysis and design phase is a critical stage in the development of any technical solution, as it defines how problems are interpreted and how solutions are structured. During this internship, multiple independent systems were developed across different domains, including simulation modeling, sports analytics, database querying, and artificial intelligence evaluation.

Unlike traditional projects that focus on a single system, this internship consists of three distinct yet conceptually related systems, each addressing a different problem domain. These systems are unified by a common objective of enabling data-driven decision-making through analytical and computational techniques. The three major systems developed are:

- Simulation and Sports Analytics System (Monte Carlo + IPL Optimization)
- SQL-Based Data Analysis System
- LLM Evaluation and Prompt Engineering System

Each system has its own inputs, processing logic, and outputs, and has been designed independently to address specific requirements. This chapter presents a detailed analysis and design of each system, including architecture, workflows, and functional components.

5.2 Overall Internship Architecture

Although the systems are independent, they follow a common high-level structure involving data collection, processing, analysis, and output generation. This unified perspective helps in understanding the overall workflow of the internship.

Fig 5.1 - Internship Architecture

At a high level, all systems begin with data input (structured data, player statistics, or

prompts), followed by preprocessing and transformation. The processed data is then analyzed using domain-specific techniques such as simulation, optimization, SQL querying, or AI evaluation. Finally, the results are presented in a structured format.

This layered architecture ensures consistency in approach while allowing flexibility in implementation.

5.3 Problem Analysis

The primary objective of this system is to predict cricket match outcomes and optimize fantasy team selection using data-driven techniques. Traditional approaches in sports analytics often rely on averages or expert opinions, which do not account for uncertainty and variability in real-world scenarios.

The challenge lies in modeling the randomness of cricket matches and evaluating players based on multiple performance metrics. Therefore, a probabilistic simulation approach combined with analytical scoring and optimization techniques was adopted.

5.4 System Design: Monte Carlo Simulation

The Monte Carlo simulation system is designed to model cricket matches using probabilistic methods. The system consists of four main components: data preprocessing, probability modeling, simulation engine, and result aggregation.

The design begins with converting historical player data into probability distributions. Each possible outcome of a ball is assigned a probability based on past performance. These probabilities are then transformed into cumulative distributions, which enable mapping of random numbers to outcomes.

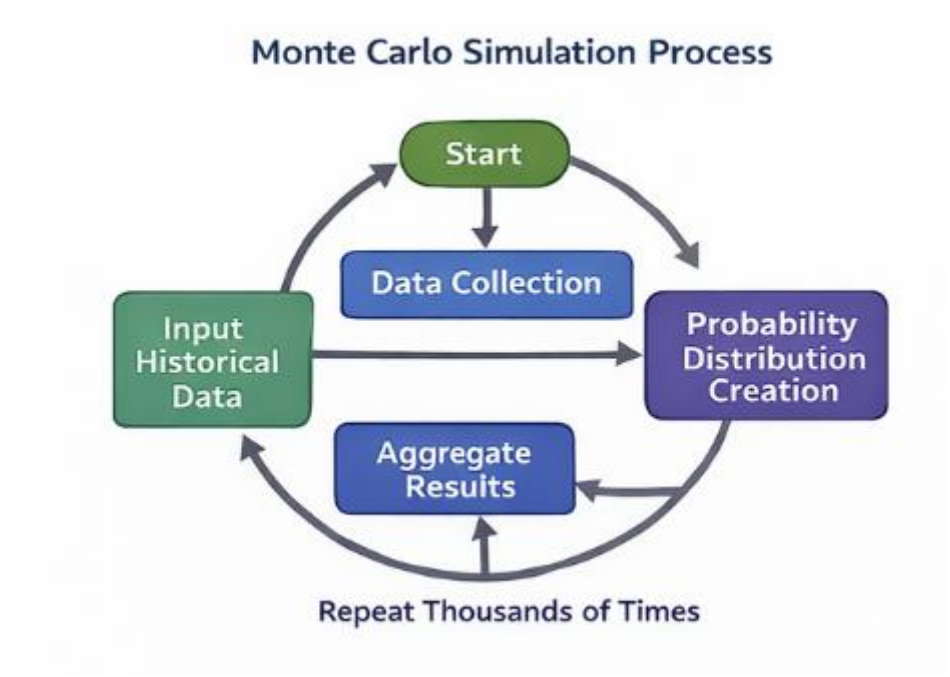


Fig 5.2 - Monte Carlo Simulation Process

The simulation engine generates random values to simulate each ball in a match. Based on the generated value and probability distribution, the outcome of the ball is determined. The match state is updated dynamically, including runs scored, wickets taken, and overs completed.

This process is repeated multiple times to generate a large number of simulated matches. The results are aggregated to calculate the probability of each team winning, providing a statistically reliable prediction.

5.5 System Design: IPL Team Optimization

The IPL team optimization system focuses on selecting the best possible team based on player performance metrics. The system includes data preprocessing, scoring model, and optimization engine.

The scoring model calculates an impact score for each player using normalized and weighted metrics. Batting metrics such as strike rate and runs are given higher importance, while bowling metrics such as wickets and economy rate are used to evaluate bowlers.

The optimization engine uses mathematical techniques such as linear programming and knapsack algorithms to select the best team within constraints such as budget, team size, and player roles.

This design ensures that the selected team maximizes performance while adhering to

practical constraints.

Fig 5.3 - IPL Team Optimization

5.6 Problem Analysis

The objective of this system is to extract meaningful insights from structured datasets using advanced SQL techniques. Traditional data retrieval methods are insufficient for handling complex analytical tasks, especially when dealing with hierarchical data and large datasets.

The challenge lies in efficiently processing data to perform operations such as ranking, aggregation, segmentation, and hierarchical analysis.

5.7 System Design: SQL Query Framework

The SQL system is designed as a multi-layered query processing framework. It includes data retrieval, transformation, and analysis components.

The system uses joins to combine data from multiple tables, enabling comprehensive analysis across datasets. Recursive queries are used to explore hierarchical relationships, such as employee reporting structures. Window functions are used to perform advanced calculations such as ranking and running totals.

The output of the system includes structured insights such as customer segmentation, sales trends, and performance rankings.

This design demonstrates the power of SQL in handling complex analytical tasks efficiently.

5.8 Problem Analysis

The Python component of the internship focuses on solving data processing and algorithmic problems using structured programming techniques. While SQL is primarily used for querying structured databases, Python provides flexibility in handling complex data structures such as dictionaries, lists, and nested objects.

The objective of this system is to process raw and semi-structured data efficiently, perform transformations, and implement logic-based solutions for analytical problems. The challenges include handling nested data structures, performing aggregations, and ensuring correctness in algorithm implementation.

5.9 System Design: Data Processing Framework

The Python system is designed to handle data in a structured and scalable manner. The design involves reading input data, iterating through nested structures, extracting relevant information, and performing computations.

For example, in the warehouse dataset problem, the data is organized in a nested dictionary format where customer data contains monthly transactions, and each transaction contains product details and amounts. The system processes this data by iterating through each level of the structure and aggregating values based on specific conditions.

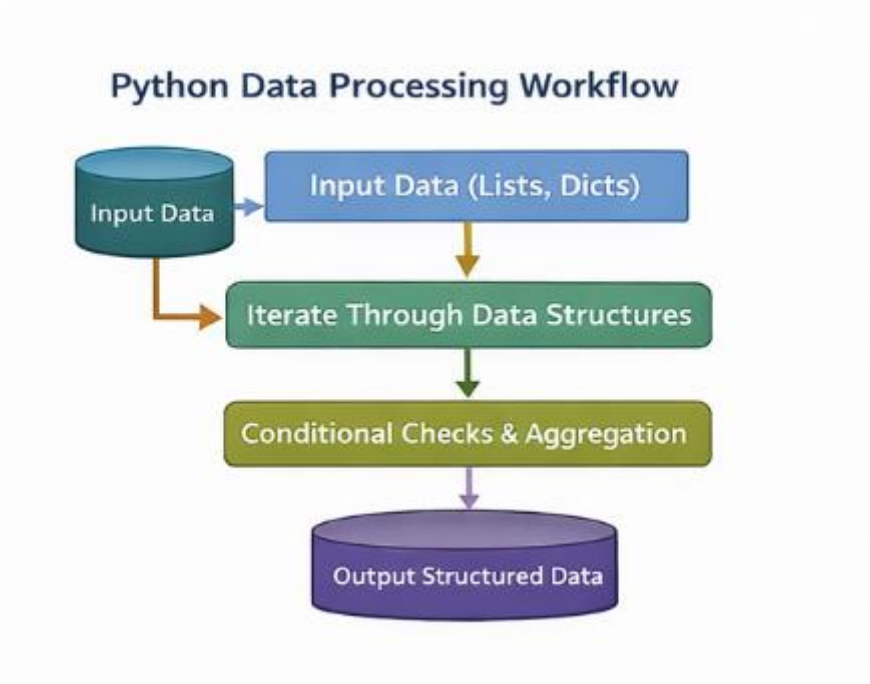


Fig 5.4 - Python Data Processing Workflow

The design emphasizes modularity, where each function performs a specific task such as data extraction, transformation, or aggregation. This approach improves readability,

maintainability, and scalability of the code.

5.10 Algorithm Design and Logic Implementation

The core of the Python system lies in implementing efficient algorithms to solve data processing problems. The methodology involves breaking down complex problems into smaller steps and using appropriate data structures to store intermediate results.

For instance, aggregation problems involve initializing a dictionary to store results and updating values dynamically as data is processed. Conditional logic is used to filter relevant data, while loops and iterations ensure that all elements are processed correctly.

During implementation, issues such as incorrect key handling or improper conditions were identified and corrected. For example, ensuring that values are stored using the correct keys and updating them appropriately prevents logical errors in aggregation.

This iterative approach to debugging and refinement ensures that the final solution is both accurate and efficient.

5.11 LLM Evaluation System Design

The LLM evaluation system is designed to assess and improve the quality of responses generated by different models. The system consists of prompt generation, rubric definition, response evaluation, ranking, and refinement modules.

The prompt generation module creates input instructions for different use cases. The rubric definition module defines evaluation criteria, which are classified into writing and non-writing quality metrics. The evaluation module assigns scores to each response based on these criteria.

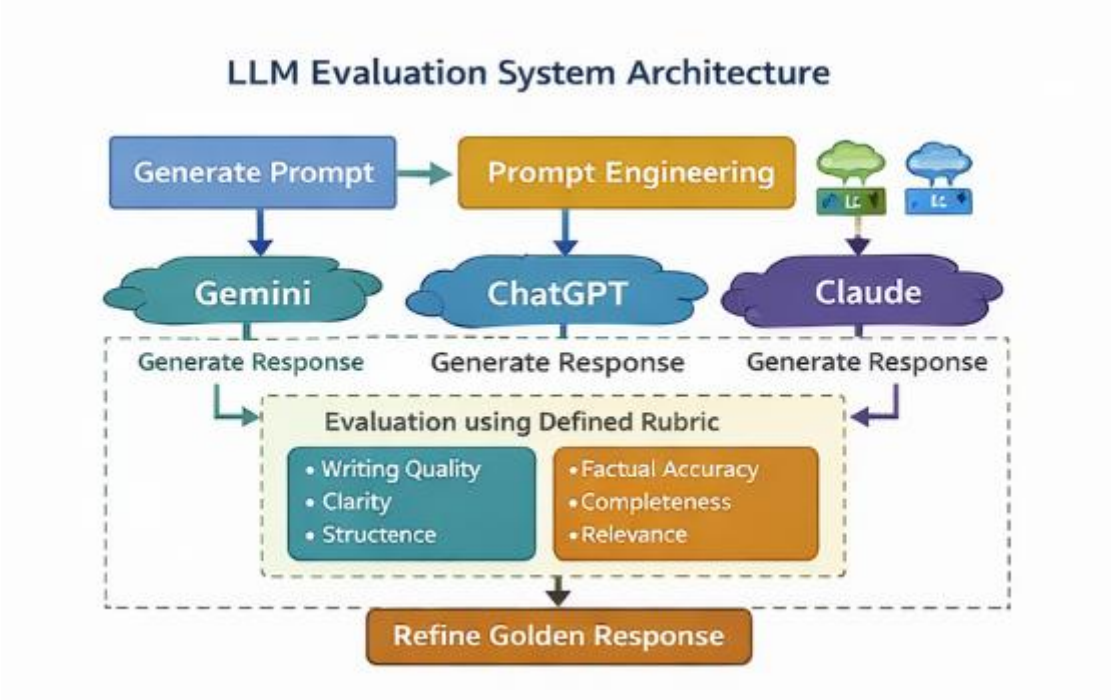


Fig 5.5 - LLM Evaluation Architecture

The ranking module compares responses and assigns ranks based on scores. The refinement module improves the best response to create a golden response that satisfies all criteria.

5.12 Summary of Analysis and Design

The analysis and design phase provided a structured framework for developing the systems implemented during the internship. By carefully analyzing requirements and designing modular architectures, the systems were able to achieve high levels of accuracy and efficiency.

The integration of simulation, analytics, database querying, and AI evaluation demonstrates the versatility of the design and its applicability to real-world problems.

5.13 Design of Python Data Processing System

The Python-based system is designed to handle hierarchical and nested datasets using a structured and modular approach. The design consists of three main components: data traversal, data extraction, and aggregation.

The data traversal component is responsible for navigating through nested data structures such as dictionaries containing lists of records. This is achieved using nested loops, where

each level of the loop corresponds to a level in the data hierarchy. The extraction component identifies relevant fields within the data, such as numerical values or specific attributes, using conditional checks.

The aggregation component processes the extracted data and computes required outputs, such as total values or frequency counts. The results are stored in a separate data structure to ensure clarity and maintainability. This modular design ensures that each component performs a specific function, improving readability and scalability of the system.

The design also incorporates error handling mechanisms to address issues such as missing keys or invalid data formats. This ensures robustness and reliability of the system when dealing with real-world datasets.

Chapter 6

Hardware, Software and Implementation

6.1 Introduction

The implementation phase focuses on the practical execution of the systems designed in the previous chapter. It involves the use of appropriate tools, programming languages, and technologies to develop working solutions for each problem domain. During this internship, multiple technologies were utilized, including Python for data processing and simulation, SQL for database analysis, and large language models for evaluation tasks.

This chapter provides a detailed explanation of the tools and technologies used, along with the implementation approach for each system. It also includes descriptions of code logic, execution flow, and output generation.

6.2 Hardware Requirements

The implementation of the projects did not require specialized hardware, as all tasks were computationally manageable on standard systems. The following hardware configuration was sufficient:

- Processor: Intel i5 or equivalent
- RAM: 8 GB or higher
- Storage: Minimum 256 GB
- Operating System: Windows / Linux / macOS

The systems were designed to be lightweight and efficient, ensuring compatibility with commonly available hardware.

6.3 Software Requirements

The following software tools and technologies were used during the internship:

- Python – Used for simulation, data processing, and algorithm implementation
- MySQL / SQL Environment – Used for database querying and analysis
- Google Sheets / Excel – Used for data collection and preprocessing
- Text Editor / IDE (VS Code / Jupyter Notebook) – Used for coding and testing

- Web Browsers – Used for accessing datasets and LLM platforms
- LLM Platforms (Gemini, ChatGPT, Claude) – Used for response generation and evaluation. These tools were selected based on their ease of use, flexibility, and suitability for analytical tasks.

6.4 Implementation of Monte Carlo Simulation

The Monte Carlo simulation was implemented using Python, where the logic was designed to simulate cricket matches ball by ball. The implementation begins with defining probability distributions for different outcomes based on historical data.

A random number generator is used to simulate each ball. The generated number is compared against cumulative probability ranges to determine the outcome, such as runs scored or a wicket. The match state is updated dynamically after each ball.

The simulation is repeated multiple times to generate a large number of match scenarios. The results are then aggregated to calculate winning probabilities.

Writing

6.5 Implementation of IPL Fantasy Team System

The IPL team selection system was implemented using Python and spreadsheet tools for data handling. The implementation begins with importing player data and preprocessing it to ensure consistency.

Normalization techniques were applied to scale different metrics into a common range. The impact score was then calculated using weighted formulas, where each metric contributes to the overall score.

Writing

Scoring Logic

Batting Score = (Strike Rate * weight1) + (Runs * weight2)

Bowling Score = (Wickets * weight3) - (Economy * weight4)

Total Score = Batting Score + Bowling Score

The optimization process involves selecting players based on maximum score while satisfying constraints such as budget and team composition.

This implementation demonstrates how data analytics can be used to make objective decisions in sports.

6.6 Implementation of Python Data Processing

The Python data processing tasks involved handling nested data structures and performing aggregations. The implementation uses loops and conditional statements to extract and process relevant information.

For example, in the warehouse dataset problem, the data is stored in nested dictionaries. The implementation iterates through each level of the dictionary and aggregates values based on specific keys.

Writing

6.7 Implementation of SQL Analysis

The SQL component was implemented using a relational database system. Queries were written to perform data retrieval, aggregation, and analysis.

Basic queries were used for filtering and selection, while advanced queries used joins, subqueries, recursive CTEs, and window functions.

For example, recursive queries were used to analyze hierarchical data, while window functions were used to calculate rankings and running totals.

Writing

Example SQL Query

```
SELECT customerNumber, SUM(amount)
FROM payments
GROUP BY customerNumber;
```

More complex queries involved multiple joins and analytical functions to extract deeper insights from the data .

[INSERT SCREENSHOT 5: SQL Query Output]

The implementation demonstrates the power of SQL in handling structured data efficiently.

6.8 Implementation of LLM Evaluation System

The LLM evaluation system was implemented by designing prompts and evaluating responses from different models. The process begins with defining a use case and creating a prompt.

Responses are generated from multiple models, including Gemini, ChatGPT, and Claude. Each response is evaluated using predefined rubrics.

Writing quality metrics are scored on a scale from 0 to 1, while non-writing quality metrics are evaluated as true or false.

Writing

Example Evaluation:

Clarity → 1

Accuracy → True

Completeness → True

Structure → 0.8

Based on these scores, responses are ranked, and the best response is refined to create a golden response.

This implementation highlights the importance of structured evaluation in improving AI-generated outputs.

6.9 Implementation of Python Logic

The implementation of the Python notebook is based on iterative processing and conditional logic. The code uses loops to traverse through nested data structures and extract relevant information. Dictionaries are used extensively to store intermediate and final results.

The implementation begins by initializing an empty dictionary to store aggregated results. As the data is processed, values are updated dynamically using key-value pairs. Conditional statements are used to identify specific attributes within the data, such as transaction amounts or product details.

One of the key aspects of the implementation is handling nested loops efficiently. Each loop is carefully structured to ensure that all levels of the data hierarchy are processed correctly. Additionally, care is taken to update values appropriately, avoiding overwriting or incorrect aggregation.

The implementation also includes debugging and refinement steps, where logical errors are identified and corrected. This iterative development process ensures that the final output is accurate and reliable.

6.9 Summary of Implementation

The implementation phase demonstrates the practical application of theoretical concepts using appropriate tools and technologies. Each system was successfully implemented using a

combination of programming, database querying, and analytical techniques.

The use of Python enabled flexible data processing and simulation, SQL facilitated efficient data analysis, and LLM platforms provided insights into AI evaluation. Together, these implementations showcase a comprehensive approach to solving real-world problems.

Chapter 7

Results and Evaluation

7.1 Introduction

The results and evaluation phase focuses on analyzing the outputs generated by the implemented systems and assessing their performance based on predefined objectives. This chapter presents the outcomes of all four systems developed during the internship, namely the Monte Carlo simulation system, IPL fantasy team optimization system, SQL-based data analysis system, and LLM evaluation framework.

The evaluation is carried out using both quantitative and qualitative approaches. Quantitative evaluation includes numerical results such as probabilities, scores, and rankings, while qualitative evaluation focuses on the effectiveness, accuracy, and reliability of the outputs. This comprehensive evaluation ensures that the systems meet their intended goals and perform efficiently under different scenarios.

7.2 Results of Monte Carlo Simulation

The Monte Carlo simulation system was evaluated by running multiple match simulations using historical probability distributions. Each match was simulated thousands of times to ensure statistical reliability.

The output of the simulation is represented in terms of winning probabilities for each team. For example, in a simulated match scenario, the system generated the following results:

Overview		
Team Name	CSK	SRH
Win Count	188	318
Result	SRH	

Overall KPI's		
	CSK	SRH
Total Runs	76162	84557
Total Wickets Taken	3052	3008
Maximum Runs	238	241
Maximum Wickets	10.00	10.00
Win Percentage	37.15%	62.85%

Batting KPI's		
Count of 4's	6297	7157
Count of 6's	3803	4620

Bowling KPI's		
Total Balls Bowled	61522	62475
Good	56744	57040
No Ball	1726	2427
Wicket	3052	3008

Fig 7.1 - Monte Carlo KPI Cards

These results indicate that Team RCB has a higher probability of winning based on the given data. The probabilistic nature of the model allows for a more realistic representation of match outcomes compared to deterministic approaches.

A key observation from the simulation is that the results stabilize as the number of simulations increases. This demonstrates the reliability of the Monte Carlo method in producing consistent outcomes.

7.2 Results of IPL Fantasy Team Selection

The IPL fantasy team system was evaluated based on its ability to select an optimal team using player performance data. The system calculated impact scores for each player and selected a team that maximized overall performance while satisfying constraints.

7.2.1 Case Study: Player Evaluation

A detailed evaluation was conducted for a sample player to understand the scoring mechanism.

Example:

- Raw metrics: Runs, strike rate, wickets
- Normalized metrics: Converted to scale (0–1)

- Weighted score: Calculated using predefined weights

The results demonstrate that normalization ensures fair comparison across players, while weighting allows prioritization of important metrics.

7.2.2 Final Team Output

The system generated an optimized team that satisfies:

- Budget constraints
- Team composition rules
- Maximum performance score

Fig 7.2 - IPL Full Squad

The selected team achieved a significantly higher total score compared to randomly selected teams, demonstrating the effectiveness of the optimization model.

7.3 Results of SQL Data Analysis

SQL-based data analysis is grounded in relational database theory, which is based on the concept of organizing data into tables with defined relationships. The relational model allows for efficient storage, retrieval, and manipulation of data using structured queries.

One of the key theoretical concepts in SQL is the use of set operations. SQL queries operate

on sets of data rather than individual records, enabling efficient processing of large datasets. Aggregation functions, such as SUM and COUNT, allow for summarizing data, while grouping operations enable analysis across different categories.

The results demonstrate the ability of SQL queries to efficiently process and aggregate large datasets. Advanced queries using window functions and recursive CTEs provided deeper insights, such as ranking customers and analyzing hierarchical data structures.

7.4 Results of LLM Evaluation System

The LLM evaluation system was tested by generating responses from multiple models and evaluating them using predefined rubrics.

7.4.1 Response Ranking

Based on the evaluation scores, responses were ranked from best to worst. The ranking process ensures that the most accurate and well-structured response is selected.

7.4.2 Golden Response Generation

The best-performing response was further refined to create a “golden response” that satisfies all evaluation criteria. This process involved improving clarity, correcting errors, and ensuring completeness.

gemini 3 vs gpt5.2	gpt5.2 vs opus4.6	gemini 3 vs opus4.6	Golden Response The perfect, masterful response. You may create the golden response(perfect response) by revising the Gemini response or creating golden response from scratch. DO NOT use or reference non-gemini model response to craft golden response The perfect response should pass all rubrics (both generic and use-case-specific rubrics).	Is rubricTrue se for Golden Response
			+----------+ | KeyGen (CPA side) -> public encaps key includes h, s, G; | | secret decaps key includes (x,y) branch | +----------+ | | v v +----------+ +----------+ | Encaps (FO + CPA) | | Decaps (FO + CPA) | | forms (u,v) layer | network | RMRS decode on | | plus FO steps not | ----------> | (v - u*y), then FO | | fully specified | ciphertext | steps not fully | | in IR 8545 | | listed in IR 8545 | +----------+ +----------+ _____/ shared secret definition deferred to the eventual KEM standard (17,18)	TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
ChatGPT is stro	ChatGPT wins o	Claude is strong	6. Stepwise logic map (plain English, not normative bytes) Key generation: Sample secret (x, y) and public randomness h; form s = x + h*y; publish h, s, and the public RMRS ge CPA encryption branch: Choose low-weight noise terms and message m; compute u = r1 + hr2 and v = mG + s*r2 + e CPA decryption branch: From (u, v) and (x, y), form (v - u*y) and run the RMRS decoder to recover the message bran FO promotion: IR 8545 states FO promotes IND-CPA PKE to IND-CCA2 KEM and ties the proof template to low DFR, 7. Illustrative trace (conceptual only, not normative test vectors) Alice publishes an encapsulation key that includes h, s, and G. Bob runs encapsulation: he samples the low-weight ra 8. Complexity snapshot from IR 8545 (x86_64 kcycles; Open Quantum Safe source noted in the report) Candidate (Level I style row in report) Key generation (kcycles) Decapsulation (kcycles) Source page BIKE Level I 637 | 1428| (13) HQC hqc-128| 105| 360| (13)	TRUE

Analyze this data

Fig 7.3 - LLM Golden Response

This step demonstrates the importance of iterative refinement in achieving high-quality outputs.

7.6 Comparative Analysis

A comparative analysis was conducted across all systems to evaluate their effectiveness.

Table 7.1 : System Comparison

System	Strength	Limitation
Monte Carlo	Handles uncertainty	Requires large simulations
IPL Analytics	Data-driven decisions	Dependent on data quality
SQL	Efficient data processing	Limited to structured data
LLM Evaluation	Improves AI outputs	Requires manual scoring

The comparison highlights that each system has its own strengths and limitations, and their combined use provides a comprehensive analytical framework.

7.7 Evaluation of Python Data Processing

The Python-based data processing system was evaluated based on its ability to correctly process nested datasets and generate accurate outputs. The results demonstrate that the system efficiently handles hierarchical data and performs aggregation tasks accurately.

The use of dictionaries for storing results ensures fast access and updates, while nested loops enable comprehensive traversal of the dataset. The system successfully processes complex data structures and produces outputs that match expected results.

One of the key observations is that the efficiency of the system depends on the structure of the data and the design of the loops. Properly structured loops and conditions significantly improve performance and reduce computational overhead.

Chapter 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

8.1 Social Aspects

The systems developed during this internship have significant social implications, particularly in the domains of sports analytics and artificial intelligence. The use of data-driven approaches in cricket analytics, such as Monte Carlo simulation and IPL team optimization, enhances decision-making by reducing reliance on subjective judgment. This promotes fairness and transparency in player evaluation and team selection, which can positively influence how sports analytics is perceived and utilized.

At the same time, the increasing reliance on data and automation may reduce the role of human intuition and expertise. While data-driven models provide objectivity, they may overlook qualitative factors such as player morale, leadership qualities, and situational awareness. Therefore, it is important to maintain a balance between analytical insights and human judgment.

The LLM evaluation system also has social relevance, as it contributes to improving the quality of AI-generated content. By ensuring that responses are accurate, clear, and complete, such systems can enhance user trust in artificial intelligence. However, widespread use of AI systems may lead to over-dependence on automated responses, which highlights the need for responsible usage and critical evaluation of AI outputs.

8.2 Legal Aspects

The development and use of data-driven systems involve several legal considerations, particularly related to data usage and intellectual property. The datasets used in this internship, such as cricket statistics, must be sourced from reliable and authorized platforms.

It is important to ensure that data is used in compliance with applicable terms and conditions to avoid legal violations.

In the context of SQL-based systems and data processing, data privacy and protection are critical. Even though the datasets used in this project are publicly available, similar systems applied in real-world scenarios may involve sensitive user data. In such cases, compliance with data protection regulations, such as data privacy laws, is essential.

For the LLM evaluation system, legal aspects include the responsible use of AI-generated content. Ensuring that generated responses do not violate copyright or produce misleading information is crucial. Proper attribution of sources and avoidance of plagiarism are important considerations when working with AI systems.

8.3 Ethical Aspects

Ethical considerations play a vital role in the development and deployment of analytical and AI-based systems. One of the primary ethical concerns in data analytics is bias. If the input data contains biases, the output generated by the system may also be biased, leading to unfair or inaccurate results. In the IPL analytics system, for example, reliance solely on statistical metrics may disadvantage players who contribute in less quantifiable ways.

Transparency is another important ethical aspect. Users should have a clear understanding of how the system generates its outputs. In simulation models, this includes explaining how probabilities are calculated and how predictions are made. Similarly, in the LLM evaluation system, the criteria used for scoring responses should be clearly defined and consistently applied.

Accountability is also essential, particularly in AI systems. Decisions based on analytical models should not be blindly accepted without verification. Human oversight is necessary to ensure that outputs are interpreted correctly and used responsibly.

8.4 Sustainability Aspects

Sustainability in computational systems refers to efficient use of resources and minimizing

environmental impact. The systems developed during this internship are designed to be computationally efficient and do not require high-end infrastructure, making them accessible and sustainable.

However, certain processes, such as large-scale Monte Carlo simulations and AI model evaluations, can be computationally intensive. Running a large number of simulations or processing extensive datasets consumes processing power and energy. Therefore, optimizing algorithms and limiting unnecessary computations are important for maintaining sustainability.

Additionally, the use of lightweight tools such as Python and SQL contributes to sustainability by enabling efficient data processing without excessive resource consumption. Future improvements could include optimizing code and leveraging more efficient algorithms to further reduce computational overhead.

8.5 Safety Aspects

Safety considerations in this internship primarily relate to the reliability and correctness of system outputs. Incorrect predictions or analysis results can lead to poor decision-making, especially in real-world applications. Therefore, ensuring accuracy through validation and testing is critical.

In the Monte Carlo simulation, safety is ensured by running multiple iterations and verifying the consistency of results. In the IPL analytics system, validation of player data and scoring mechanisms helps prevent incorrect team selection. For SQL queries, careful design and testing ensure that data is processed correctly without errors.

CONCLUSION

The internship provided a comprehensive and practical exposure to multiple domains within computer science, including simulation modeling, data analytics, database management, and artificial intelligence evaluation. The work carried out during this period successfully demonstrated the application of theoretical concepts to real-world problems, thereby bridging the gap between academic knowledge and practical implementation.

One of the major achievements of the internship was the development of a Monte Carlo simulation model for predicting cricket match outcomes. This system effectively utilized probabilistic techniques to model uncertainty and generate realistic predictions. By simulating thousands of match scenarios, the system was able to provide statistically reliable insights into match outcomes, highlighting the advantages of simulation-based approaches over traditional deterministic methods.

The IPL fantasy team selection system further demonstrated the importance of data-driven decision-making. By analyzing player performance metrics and applying normalization and optimization techniques, the system was able to generate an optimal team that maximizes overall performance while satisfying practical constraints. This project emphasized the role of mathematical modeling and analytical thinking in solving complex decision-making problems.

In addition to analytics and simulation, the internship involved extensive work in SQL-based data analysis. Advanced querying techniques such as joins, window functions, and recursive queries were used to extract meaningful insights from structured datasets. This component of the internship highlighted the importance of efficient data management and the power of SQL in handling large-scale analytical tasks.

The Python-based data processing tasks provided valuable experience in handling nested and semi-structured data. By implementing logical algorithms and data traversal techniques, complex data processing problems were solved efficiently. This demonstrated the flexibility and effectiveness of Python as a programming language for analytical applications.

A particularly significant aspect of the internship was the evaluation of large language models. By designing prompts, defining evaluation rubrics, and systematically scoring

responses, a structured framework was developed for assessing AI-generated outputs. The process of generating a "golden response" further highlighted the importance of iterative refinement and optimization in achieving high-quality results.

Overall, the internship successfully achieved its objectives by integrating multiple technologies and methodologies into a cohesive framework. The experience gained has significantly enhanced technical skills, problem-solving abilities, and understanding of real-world applications. The knowledge acquired during this internship will serve as a strong foundation for future academic and professional endeavors.

REFERENCES

[1] S. M. Ross, *Introduction to Probability Models*, 11th ed. Academic Press, 2014.

[2] G. S. Fishman, *Monte Carlo: Concepts, Algorithms, and Applications*. Springer, 1996.

[3] A. M. Law and W. D. Kelton, *Simulation Modeling and Analysis*, 3rd ed. McGraw-Hill, 2000.

[4] D. C. Montgomery and G. C. Runger, *Applied Statistics and Probability for Engineers*, 5th ed. Wiley, 2010.

[5] W. L. Winston, *Operations Research: Applications and Algorithms*, 4th ed. Cengage Learning, 2004.

[6] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. McGraw-Hill, 2019.

[7] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[8] C. J. Date, *An Introduction to Database Systems*, 8th ed. Addison-Wesley, 2003.

[9] W. McKinney, *Python for Data Analysis*, 2nd ed. O'Reilly Media, 2018.

[10] G. van Rossum and F. L. Drake, *Python 3 Reference Manual*. CreateSpace, 2009.

[11] M. Lutz, *Learning Python*, 5th ed. O'Reilly Media, 2013.

[12] L. Ramalho, *Fluent Python*. O'Reilly Media, 2015.

[13] F. Chollet, *Deep Learning with Python*. Manning Publications, 2017.

[14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[15] T. B. Brown *et al.*, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[16] OpenAI, "GPT Models Documentation," 2023. [Online]. Available: https://platform.openai.com

[17] Google, "Gemini AI Documentation," 2023. [Online]. Available: https://ai.google

[18] Anthropic, "Claude AI Documentation," 2023. [Online]. Available: https://www.anthropic.com

[19] Oracle, "MySQL Reference Manual," 2023. [Online]. Available: https://dev.mysql.com/doc

[20] "Pandas User Guide," Pandas Documentation, 2023. [Online]. Available: https://pandas.pydata.org

[21] "NumPy Reference Guide," NumPy Documentation, 2023. [Online]. Available: https://numpy.org

[22] "IPL Official Website: Player Statistics and Match Data," 2023. [Online]. Available: https://www.iplt20.com

[23] "ESPN Cricinfo: Cricket Statistics and Records," 2023. [Online]. Available: https://www.espncricinfo.com

APPENDIX - A

IPL Full Team Selection LPP Code

Selection of Full Squad:

```
# ========================
# FULL SQUAD SELECTION (18 PLAYERS)
# ========================

from pulp import (
  LpProblem, LpVariable, LpMaximize,
  lpSum, LpBinary, LpStatus
)

# -------------------------
# ASSUMPTION: df already exists
# Columns:
# Player_Name, type,
# Final_Batsman_Score, Final_Bowler_Score,
# Final_all_score, Final_wkt_score,
# Sold_Price
# -------------------------

# -------------------------
# STEP 1: Create capability flags (CRITICAL)
# -------------------------
df["Is_Batsman"] = df["Final_Batsman_Score"] > 0
df["Is_Bowler"] = df["Final_Bowler_Score"] > 0
df["Is_AllRounder"] = df["Final_all_score"] > 0
df["Is_WicketKeeper"] = df["Final_wkt_score"] > 0

# -------------------------
# STEP 2: Create unified player score
# -------------------------
df["Final_Player_Score"] = df[
```

```
  [
    "Final_Batsman_Score",
    "Final_Bowler_Score",
    "Final_all_score",
    "Final_wkt_score"
  ]
].fillna(0).max(axis=1)

# -------------------------
# STEP 3: Define LP model
# -------------------------
model = LpProblem(
  "IPL_Full_18_Squad_Selection",
  LpMaximize
)

players = df.index

# Decision variable: select player or not
select = {
  i: LpVariable(f"select_{i}", cat=LpBinary)
  for i in players
}

# -------------------------
# STEP 4: Objective function
# -------------------------
model += lpSum(
  select[i] * df.loc[i, "Final_Player_Score"]
  for i in players
)

# -------------------------
# STEP 5: Budget constraint (₹125 cr)
# -------------------------
model += lpSum(
  select[i] * df.loc[i, "Sold_Price"]
  for i in players
```

```
) <= 125

# -------------------------
# STEP 6: Squad size = 18
# -------------------------
model += lpSum(select[i] for i in players) == 18

# -------------------------
# STEP 7: Role constraints (FEASIBLE VERSION)
# -------------------------

# Wicket Keepers: exactly 2
model += lpSum(
  select[i] * df.loc[i, "Is_WicketKeeper"]
  for i in players
) == 2

# Batting-capable players: 4–6
model += lpSum(
  select[i] * df.loc[i, "Is_Batsman"]
  for i in players
) >= 4

model += lpSum(
  select[i] * df.loc[i, "Is_Batsman"]
  for i in players
) <= 6

# Bowling-capable players: 4–6
model += lpSum(
  select[i] * df.loc[i, "Is_Bowler"]
  for i in players
) >= 4

model += lpSum(
  select[i] * df.loc[i, "Is_Bowler"]
  for i in players
) <= 6
```

```
# All-rounders: 3–4
model += lpSum(
   select[i] * df.loc[i, "Is_AllRounder"]
   for i in players
) >= 3

model += lpSum(
   select[i] * df.loc[i, "Is_AllRounder"]
   for i in players
) <= 4

# -------------------------
# STEP 8: Solve the LP
# -------------------------
model.solve()

print("Full Squad Status:", LpStatus[model.status])

# -------------------------
# STEP 9: Extract the 18-player squad
# -------------------------
full_squad = df.loc[
   [i for i in players if select[i].value() == 1]
]

# -------------------------
# STEP 10: Sanity checks
# -------------------------
print("Total Players Selected:", len(full_squad))
print("Total Budget Used:", full_squad["Sold_Price"].sum())

print("\nRole Counts:")
print("Batsmen-capable:", full_squad["Is_Batsman"].sum())
print("Bowlers-capable:", full_squad["Is_Bowler"].sum())
print("All-rounders:", full_squad["Is_AllRounder"].sum())
print("Wicket Keepers:", full_squad["Is_WicketKeeper"].sum())
```

```
# -------------------------
# STEP 11: View final 18-man squad
# -------------------------
full_squad[
  ["Player_Name", "type", "Final_Player_Score", "Sold_Price"]
].sort_values(
  "Final_Player_Score", ascending=False
)
```

This model uses binary linear programming to select an 18-member IPL squad under a fixed ₹125 crore budget. Players are evaluated using role-specific performance scores, consolidated into a unified score. Role constraints are applied using capability flags rather than rigid labels, allowing realistic overlap between batsmen, bowlers, all-rounders, and wicket keepers.

Selection of Dream 11 Players:

```
# ========================
# DREAM 11 SELECTION (FROM FULL SQUAD)
# ========================

from pulp import (
  LpProblem, LpVariable, LpMaximize,
  lpSum, LpBinary, LpStatus
)

# -------------------------
# ASSUMPTION:
# full_squad already exists from previous step
# Columns present:
# Player_Name, type,
# Final_Player_Score,
# Final_Batsman_Score, Final_Bowler_Score,
# Final_all_score, Final_wkt_score
# -------------------------
```

```
# -------------------------
# STEP 1: Create Dream 11 LP model
# -------------------------
xi_model = LpProblem(
  "Dream_11_Playing_XI_Selection",
  LpMaximize
)

players = full_squad.index

# Decision variable: play in XI or not
play = {
  i: LpVariable(f"play_{i}", cat=LpBinary)
  for i in players
}

# -------------------------
# STEP 2: Objective function
# Maximize on-field performance
# -------------------------
xi_model += lpSum(
  play[i] * full_squad.loc[i, "Final_Player_Score"]
  for i in players
)

# -------------------------
# STEP 3: Playing XI size = 11
# -------------------------
xi_model += lpSum(play[i] for i in players) == 11

# -------------------------
# STEP 4: Dream 11 role constraints
# (Realistic IPL balance)
# -------------------------

# At least 1 Wicket Keeper
xi_model += lpSum(
  play[i] * (full_squad.loc[i, "Final_wkt_score"] > 0)
```

```
  for i in players
) >= 1

# At least 3 batsmen
xi_model += lpSum(
  play[i] * (full_squad.loc[i, "Final_Batsman_Score"] > 0)
  for i in players
) >= 3

# At least 2 all-rounders
xi_model += lpSum(
  play[i] * (full_squad.loc[i, "Final_all_score"] > 0)
  for i in players
) >= 2

# At least 3 bowlers
xi_model += lpSum(
  play[i] * (full_squad.loc[i, "Final_Bowler_Score"] > 0)
  for i in players
) >= 3

# -------------------------
# STEP 5: Solve Dream 11 LP
# -------------------------
xi_model.solve()

print("Dream 11 Status:", LpStatus[xi_model.status])

# -------------------------
# STEP 6: Extract Dream 11
# -------------------------
dream_11 = full_squad.loc[
  [i for i in players if play[i].value() == 1]
]

# -------------------------
# STEP 7: Sanity checks
# -------------------------
```

```
print("Total Players in Dream 11:", len(dream_11))

print("\nRole Counts in Dream 11:")
print("Batsmen:", (dream_11["Final_Batsman_Score"] > 0).sum())
print("Bowlers:", (dream_11["Final_Bowler_Score"] > 0).sum())
print("All-rounders:", (dream_11["Final_all_score"] > 0).sum())
print("Wicket Keepers:", (dream_11["Final_wkt_score"] > 0).sum())

# -------------------------
# STEP 8: View Final Dream 11
# -------------------------
dream_11[
  ["Player_Name", "type", "Final_Player_Score"]
].sort_values(
  "Final_Player_Score", ascending=False
)
```

After selecting the optimal 18-member squad under budget constraints, a second binary linear program was formulated to select the best playing XI. The model maximizes on-field performance while enforcing realistic role balance using capability-based constraints, allowing wicket keeper–batsmen and all-rounders to fulfill multiple roles.

Classic Models Dataset - Advance SQL Queries

1.Find the total number of employees reporting under each manager recursively.

Approach 1: Top-Down Recursive CTE

```
WITH RECURSIVE emp_hierarchy AS (
   SELECT reportsTo AS manager_id, employeeNumber AS employee_id
   FROM employees
   WHERE reportsTo IS NOT NULL
   UNION ALL
   SELECT eh.manager_id, e.employeeNumber
   FROM emp_hierarchy eh
   JOIN employees e ON e.reportsTo = eh.employee_id
)
SELECT manager_id, COUNT(employee_id) AS Total_emp_under_manager
FROM emp_hierarchy
GROUP BY manager_id;
```

Explanation: This approach uses a recursive CTE to traverse the hierarchy from the manager downwards to find all direct and indirect subordinates.
It effectively links every employee to their respective ancestor manager in the chain for accurate counting.

Approach 2: Bottom-Up Recursive CTE

```
WITH RECURSIVE manager_chain AS (
   SELECT employeeNumber AS employee_id, reportsTo AS manager_id
   FROM employees
   WHERE reportsTo IS NOT NULL
   UNION ALL
   SELECT mc.employee_id, e.reportsTo
   FROM manager_chain mc
   JOIN employees e ON mc.manager_id = e.employeeNumber
   WHERE e.reportsTo IS NOT NULL
)
SELECT manager_id, COUNT(employee_id) AS total_employees
```

```
FROM manager_chain
GROUP BY manager_id;
```

Explanation: This method starts with individual employees and recursively moves upward to identify every manager in their reporting line.
By grouping by the higher-level managers found in the chain, it tallies the total workforce managed by each leader.

#	manager_id	Total_emp_under_manager
1	1002	22
2	1056	20
3	1088	3
4	1102	6
5	1143	6
6	1621	1

2. Identify the maximum depth of the organizational hierarchy.

Approach 1: Recursive CTE Depth Tracking

```
WITH RECURSIVE CTE_depth AS (
   SELECT employeeNumber, reportsTo, 1 AS level
   FROM employees WHERE reportsTo IS NULL
   UNION ALL
   SELECT e.employeeNumber, e.reportsTo, level+1
   FROM employees e JOIN CTE_depth cd
   ON cd.employeeNumber = e.reportsTo
)
SELECT MAX(level) AS Hierarchy_depth FROM CTE_depth;
```

Explanation: This query assigns a numerical level to each employee starting from the top executive (level 1) and incrementing downwards.
The final MAX(level) identifies the deepest point in the organizational structure.

Approach 2: Fixed-Level Self-Joins

```
SELECT MAX(depth) AS max_depth
FROM (
   SELECT e1.employeeNumber, 1 +
```

```
    (e2.employeeNumber IS NOT NULL) +
    (e3.employeeNumber IS NOT NULL) +
    (e4.employeeNumber IS NOT NULL) +
    (e5.employeeNumber IS NOT NULL) AS depth
  FROM employees e1
  LEFT JOIN employees e2 ON e2.reportsTo = e1.employeeNumber
  LEFT JOIN employees e3 ON e3.reportsTo = e2.employeeNumber
  LEFT JOIN employees e4 ON e4.reportsTo = e3.employeeNumber
  LEFT JOIN employees e5 ON e5.reportsTo = e4.employeeNumber
) t;
```

Explanation: This approach uses a series of LEFT JOIN operations to manually check for subsequent levels in the hierarchy.
It is a non-recursive alternative suitable for databases with a known or limited maximum hierarchy depth.

#	Hierarchy_depth
1	4

3.Create a recursive query to display each employee along with the full chain of offices they belong to.

Approach 1: Recursive CTE (Management Chain Mapping)

```
WITH RECURSIVE office_chain AS (
    SELECT
        e.employeeNumber,
        e.reportsTo,
        CAST(o.city AS CHAR(1000)) AS office_path
    FROM employees e
    JOIN offices o ON e.officeCode = o.officeCode

    UNION ALL

    SELECT
        oc.employeeNumber,
        m.reportsTo,
        CONCAT(oc.office_path, ' -> ', o.city)
    FROM office_chain oc
    JOIN employees m ON oc.reportsTo = m.employeeNumber
    JOIN offices o ON m.officeCode = o.officeCode
)
SELECT employeeNumber, office_path
FROM office_chain
```

```
WHERE reportsTo IS NULL;
```

Explanation: This approach uses a Recursive CTE to traverse the management hierarchy, dynamically appending office cities to a path string as it moves up the chain.

The CAST function is used in the anchor member to pre-allocate character length, preventing the "Data too long" error as the string grows during recursion.

Approach 2: Multiple Self-Joins (Fixed-Depth Path)

```
SELECT
   e1.employeeNumber,
   CAST(CONCAT(o1.city,
         COALESCE(CONCAT(' -> ', o2.city), ''),
         COALESCE(CONCAT(' -> ', o3.city), ''),
         COALESCE(CONCAT(' -> ', o4.city), '')) AS CHAR(1000)) AS office_chain
FROM employees e1
JOIN offices o1 ON e1.officeCode = o1.officeCode
LEFT JOIN employees e2 ON e1.reportsTo = e2.employeeNumber
LEFT JOIN offices o2 ON e2.officeCode = o2.officeCode
LEFT JOIN employees e3 ON e2.reportsTo = e3.employeeNumber
LEFT JOIN offices o3 ON e3.officeCode = o3.officeCode
LEFT JOIN employees e4 ON e3.reportsTo = e4.employeeNumber
LEFT JOIN offices o4 ON e4.officeCode = o4.officeCode;
```

Explanation: This method uses a series of LEFT JOIN operations to manually link employees to their managers' offices up to a specific depth in the hierarchy.

By wrapping the CONCAT function in a CAST, the query ensures the output column is wide enough to store the combined names of multiple cities without truncation.

#	employeeNumber	office_chain
1	1188	Boston -> San Francisco -> San Francisco -> San Francisco
2	1216	Boston -> San Francisco -> San Francisco -> San Francisco
3	1501	London -> Paris -> San Francisco -> San Francisco
4	1504	London -> Paris -> San Francisco -> San Francisco
5	1286	NYC -> San Francisco -> San Francisco -> San Francisco
6	1323	NYC -> San Francisco -> San Francisco -> San Francisco
7	1337	Paris -> Paris -> San Francisco -> San Francisco

4.For each customer, calculate the running total of their payments using SUM as a window function.

Approach 1: SUM() Window Function

```
SELECT c.customerNumber, c.customerName, p.amount,
SUM(p.amount) OVER(PARTITION BY customerNumber ORDER BY paymentDate
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
running_total
FROM customers c
JOIN payments p ON c.customerNumber = p.customerNumber;
```

Explanation: This utilizes the standard SQL OVER clause to calculate a cumulative sum for each customer over time.
It is highly efficient and automatically handles the windowing for each payment record.

Approach 2: Correlated Subquery

```
SELECT c.customerNumber, c.customerName, p.paymentDate, p.amount,
(SELECT SUM(p2.amount) FROM payments p2 WHERE p2.customerNumber =
p.customerNumber AND p2.paymentDate <= p.paymentDate) AS running_total
FROM customers c
JOIN payments p ON c.customerNumber = p.customerNumber
ORDER BY c.customerNumber, p.paymentDate;
```

Explanation: This method calculates the running total by running a separate sum for every row that counts all previous payments.
While accurate, it can be significantly slower than the window function approach on larger datasets.

#	customerNumber	customerName	amount	running_tota
1	103	Atelier graphique	14571.44	14571.44
2	103	Atelier graphique	6066.78	20638.22
3	103	Atelier graphique	1676.14	22314.36
4	112	Signal Gift Stores	32641.98	32641.98
5	112	Signal Gift Stores	33347.88	65989.86
6	112	Signal Gift Stores	14191.12	80180.98
7	114	Australian Collectors, Co.	45864.03	45864.03
8	114	Australian Collectors, Co.	7565.08	53429.11
9	114	Australian Collectors, Co.	44894.74	98323.85
10	114	Australian Collectors, Co.	82261.22	180585.07
11	119	La Rochelle Gifts	47924.19	47924.19

5.Determine the ROW_NUMBER of orders placed by each customer, ordered by the order date.

Approach 1: ROW_NUMBER() Window Function

```
SELECT customerNumber, orderNumber, orderdate,
ROW_NUMBER() OVER(PARTITION BY customerNumber ORDER BY orderDate) AS order_rank
FROM orders;
```

Explanation: This approach assigns a sequential integer to each order for every customer based on the chronological order date.
It is the most direct and performance-optimized way to rank items within specific groups.

Approach 2: Correlated Subquery Count

```
SELECT o1.customerNumber, o1.orderNumber, o1.orderDate,
(SELECT COUNT(*) FROM orders o2 WHERE o2.customerNumber =
o1.customerNumber AND o2.orderDate <= o1.orderDate) AS order_rank
FROM orders o1
ORDER BY o1.customerNumber, o1.orderDate;
```

Explanation: This query determines the rank by counting how many orders exist for the same customer with an earlier date.

This provides a ranking logic equivalent to ROW_NUMBER without using modern window functions.

#	customerNumber	orderNumber	orderdate	order_rank
1	103	10123	2003-05-20	1
2	103	10298	2004-09-27	2
3	103	10345	2004-11-25	3
4	112	10124	2003-05-21	1
5	112	10278	2004-08-06	2
6	112	10346	2004-11-29	3
7	114	10120	2003-04-29	1
8	114	10125	2003-05-21	2
9	114	10223	2004-02-20	3
10	114	10342	2004-11-24	4
11	114	10347	2004-11-29	5

6. Use FIRST_VALUE and LAST_VALUE to identify the first and last products ordered by a specific customer.

Approach 1: Direct Window Functions

```
SELECT DISTINCT c.customerNumber, c.customerName,
FIRST_VALUE(p.productName) OVER(PARTITION BY c.customerNumber,
c.customerName ORDER BY o.orderDate) AS First_Order,
LAST_VALUE(p.productName) OVER(PARTITION BY c.customerNumber,
c.customerName ORDER BY o.orderDate ROWS BETWEEN UNBOUNDED
PRECEDING AND UNBOUNDED FOLLOWING) AS Last_Order
FROM customers c
JOIN orders o ON c.customerNumber = o.customerNumber
JOIN orderdetails od ON o.orderNumber = od.orderNumber
JOIN products p ON od.productCode = p.productCode
ORDER BY customerNumber;
```

Explanation: Uses FIRST_VALUE and LAST_VALUE to pluck the first and last product names from the sorted transaction history.

The frame UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING ensures the entire range is evaluated for each customer.

Approach 2: CTE with Window Functions

```
WITH CTE_First_Last AS (
   SELECT DISTINCT c.customerNumber, c.customerName,
   FIRST_VALUE(p.productName) OVER(PARTITION BY c.customerNumber,
c.customerName ORDER BY orderDate) AS First_Order,
   LAST_VALUE(p.productName) OVER(PARTITION BY c.customerNumber,
c.customerName ORDER BY orderDate ROWS BETWEEN UNBOUNDED PRECEDING
AND UNBOUNDED FOLLOWING) AS Last_Order
   FROM customers c
   JOIN orders o ON c.customerNumber = o.customerNumber
   JOIN orderdetails od ON o.orderNumber = od.orderNumber
   JOIN products p ON od.productCode = p.productCode
)
SELECT * FROM CTE_First_Last;
```

Explanation: This approach wraps the window function logic in a CTE to separate data preparation from final selection.

It improves code readability and allows for easier secondary filtering if needed.

#	customerNumber	customerName	First_Order	Last_Order
1	103	Atelier graphique	1966 Shelby Cobra 427 S/C	1938 Cadillac V-16 Presidential Lim…
2	112	Signal Gift Stores	1969 Chevrolet Camaro Z28	1937 Lincoln Berline
3	114	Australian Collectors, Co.	1996 Moto Guzzi 1100i	1926 Ford Fire Engine
4	119	La Rochelle Gifts	1996 Moto Guzzi 1100i	1940s Ford truck
5	121	Baane Mini Imports	1926 Ford Fire Engine	1962 City of Detroit Streetcar
6	124	Mini Gifts Distributors Ltd.	1996 Peterbilt 379 Stake …	1928 Mercedes-Benz SSK
7	128	Blauer See Auto, Co.	1932 Model A Ford J-Coupe	1940s Ford truck
8	129	Mini Wheels Co.	1934 Ford V8 Coupe	1936 Chrysler Airflow
9	131	Land of Toys Inc.	1997 BMW F650 ST	1997 BMW F650 ST
10	141	Euro+ Shopping Channel	Diamond T620 Semi-Skirt…	1996 Peterbilt 379 Stake Bed with …
11	144	Volvo Model Replicas, Co	1913 Ford Model T Speed…	1962 Volkswagen Microbus

7. Segment employees into groups based on the number of subordinates they manage.

Approach 1: LEFT JOIN with CASE

```
SELECT e.employeeNumber AS emp_id, COUNT(e2.employeeNumber) AS Managing,
CASE
   WHEN COUNT(e2.employeeNumber) = 0 THEN 'Individual Contributer'
   WHEN COUNT(e2.employeeNumber) BETWEEN 1 AND 3 THEN 'Small Team
Manager'
```

```
    ELSE 'Large Team Manager'
END AS emp_segment
FROM employees e
LEFT JOIN employees e2 ON e.employeeNumber = e2.reportsTo
GROUP BY emp_id
ORDER BY Managing DESC;
```

Explanation: Performs a self-join to link managers to their subordinates and counts the occurrences.

The CASE statement then categorizes them into clear functional groups based on the count.

Approach 2: Subquery in FROM Clause

```
SELECT e.employeeNumber AS emp_id, COALESCE(s.sub_count,0) AS Managing,
CASE
    WHEN COALESCE(s.sub_count,0) = 0 THEN 'Individual Contributor'
    WHEN COALESCE(s.sub_count,0) BETWEEN 1 AND 3 THEN 'Small Team Manager'
    ELSE 'Large Team Manager'
END AS emp_segment
FROM employees e
LEFT JOIN (
    SELECT reportsTo, COUNT(*) AS sub_count
    FROM employees WHERE reportsTo IS NOT NULL
    GROUP BY reportsTo
) s ON e.employeeNumber = s.reportsTo
ORDER BY Managing DESC;
```

Explanation: Aggregates the reporting counts first in a derived table before joining back to the main employee table.

Using COALESCE ensures that employees with no subordinates are correctly handled as having zero managed staff.

#	emp_id	Managing	emp_segment
1	1102	6	Large Team Manager
2	1143	6	Large Team Manager
3	1056	4	Large Team Manager
4	1088	3	Small Team Manager
5	1002	2	Small Team Manager
6	1621	1	Small Team Manager
7	1702	0	Individual Contributer
8	1625	0	Individual Contributer
9	1619	0	Individual Contributer
10	1612	0	Individual Contributer
11	1611	0	Individual Contributer

8. For each customer, segment their payments into quartiles and calculate the percentage of payments in each quartile.

Approach 1: Nested Aggregation with NTILE

```
SELECT customerNumber, quartile, COUNT(*) AS payments_in_quartile,
ROUND(COUNT(*) * 100.0 / SUM(COUNT(*)) OVER(PARTITION BY
customerNumber), 2) AS percentage
FROM (
   SELECT customerNumber, NTILE(4) OVER(PARTITION BY customerNumber
ORDER BY amount) AS quartile
   FROM payments
) t
GROUP BY customerNumber, quartile
ORDER BY customerNumber, quartile;
```

Explanation: Groups payments into four equal parts per customer using NTILE(4) and then calculates their frequency.

The percentage is derived using a nested window function to get the total count within the same group.

Approach 2: Multi-step CTE Approach

```
WITH quartile_data AS (
   SELECT customerNumber, NTILE(4) OVER(PARTITION BY customerNumber
ORDER BY amount) AS quartile
```

```
    FROM payments
),
total_payments AS (
    SELECT customerNumber, COUNT(*) AS total_count
    FROM payments GROUP BY customerNumber
)
SELECT q.customerNumber, q.quartile, COUNT(*) AS payments_in_quartile,
ROUND(COUNT(*) * 100.0 / t.total_count, 2) AS percentage
FROM quartile_data q
JOIN total_payments t ON q.customerNumber = t.customerNumber
GROUP BY q.customerNumber, q.quartile, t.total_count;
```

Explanation: This method breaks the logic into a ranking step and a total-count step using two CTEs.

It provides a more modular structure that is easier to debug and verify for complex percentage calculations.

#	customerNumber	quartile	payments_in_quartile	percentage
1	103	1	1	33.33
2	103	2	1	33.33
3	103	3	1	33.33
4	112	1	1	33.33
5	112	2	1	33.33
6	112	3	1	33.33
7	114	1	1	25.00
8	114	2	1	25.00
9	114	3	1	25.00
10	114	4	1	25.00
11	119	1	1	33.33

9. Determine the total quantity of products sold in each country.

Approach 1: Multi-Table JOIN

```
SELECT c.country AS Country, SUM(od.quantityOrdered) AS Quantity
FROM customers c
JOIN orders o ON c.customerNumber = o.customerNumber
```

```
JOIN orderdetails od ON o.orderNumber = od.orderNumber
GROUP BY c.country;
```

Explanation: This query uses standard joins to trace order quantities back to the customer's home country.

It is the standard relational method for aggregating metrics across multiple descriptive tables.

Approach 2: Pre-aggregated Subquery

```
SELECT c.country AS Country, SUM(t.total_qty) AS Quantity
FROM customers c
JOIN orders o ON c.customerNumber = o.customerNumber
JOIN (
   SELECT orderNumber, SUM(quantityOrdered) AS total_qty
   FROM orderdetails GROUP BY orderNumber
) t ON o.orderNumber = t.orderNumber
GROUP BY c.country;
```

Explanation: This query sums quantities at the order level first before joining to customers and countries.

This can sometimes optimize performance by reducing rows involved in the primary join operations.

#	Country	Quantity
1	France	11090
2	USA	35844
3	Australia	6246
4	Norway	1082
5	Germany	2148
6	Spain	12429
7	Sweden	2006
8	Denmark	2197
9	Singapore	2760
10	Norway	1760
11	Japan	1842

10. Find customers whose total payments exceed the average payment for all customers.

Approach 1: HAVING with Subquery

```
SELECT c.customerNumber Cust_Number, c.customerName AS Cust_Name,
SUM(p.amount) Amount
FROM customers c
JOIN payments p ON c.customerNumber = p.customerNumber
GROUP BY Cust_Number, Cust_Name
HAVING Amount > (
   SELECT AVG(total_payments) FROM (
      SELECT SUM(amount) AS total_payments FROM payments GROUP BY
customerNumber
   ) t
) ORDER BY Amount DESC;
```

Explanation: This uses a HAVING clause to filter grouped results based on a dynamically calculated global average.
It identifies high-value customers by comparing their individual sum to the mean of all customer sums.

Approach 2: CTE for Better Readability

```
WITH customer_totals AS (
   SELECT customerNumber, SUM(amount) AS total_payment
   FROM payments GROUP BY customerNumber
)
SELECT c.customerNumber AS Cust_Number, c.customerName AS Cust_Name,
ct.total_payment AS Amount
FROM customer_totals ct
JOIN customers c ON ct.customerNumber = c.customerNumber
WHERE ct.total_payment > (SELECT AVG(total_payment) FROM customer_totals)
ORDER BY Amount DESC;
```

Explanation: Pre-calculates customer totals in a CTE, making the final comparison and join logic much cleaner.

This approach is generally preferred for complex comparisons where the same aggregate value is used multiple times.

#	Cust_Numbe	Cust_Name	Amount
1	141	Euro+ Shopping Channel	715738.98
2	124	Mini Gifts Distributors Ltd.	584188.24
3	114	Australian Collectors, Co.	180585.07
4	151	Muscle Machine Inc	177913.95
5	148	Dragon Souveniers, Ltd.	156251.03
6	323	Down Under Souvenier…	154622.08
7	187	AV Stores, Co.	148410.09
8	276	Anna's Decorations, Ltd	137034.22
9	321	Corporate Gift Ideas Co.	132340.78
10	146	Saveley & Henriot, Co.	130305.35
11	278	Rovelli Gifts	127529.69

11. Calculate the average time it takes for orders to be shipped after the orderDate for each country.

Approach 1: Subquery Calculation

```
SELECT Country, ROUND(AVG(day_diff)) AS Avg_Days
FROM (
   SELECT c.country AS Country, DATEDIFF(o.shippedDate, o.orderDate) AS day_diff
   FROM orders o JOIN customers c ON o.customerNumber = c.customerNumber
) k
GROUP BY Country ORDER BY Avg_Days;
```

Explanation: This calculates the difference in days for every single order before averaging them by country.

It provides a straightforward performance metric for logistics efficiency across geographical regions.

Approach 2: CTE for Shipping Metrics

```
WITH shipping_time AS (
   SELECT c.country, DATEDIFF(o.shippedDate, o.orderDate) AS ship_days
   FROM orders o JOIN customers c ON o.customerNumber = c.customerNumber
)
```

```
SELECT country, ROUND(AVG(ship_days)) AS Avg_Days
FROM shipping_time GROUP BY country ORDER BY Avg_Days;
```

Explanation: By defining shipping_time as a CTE, the query separates time-calculation from the logic of regional averaging.
This makes it easier to add further filters, such as excluding orders that haven't shipped yet.

#	Country	Avg_Days
1	Switzerland	2
2	Denmark	2
3	Belgium	2
4	Germany	3
5	Hong Kong	3
6	Finland	3
7	New Zeal...	3
8	Austria	3
9	UK	3
10	Sweden	4
11	France	4

12. Determine the top 5 products with the highest total sales (quantityOrdered * priceEach).

Approach 1: Standard Aggregation and Limit

```
SELECT p.productName AS Product_Name, SUM(od.quantityOrdered * od.priceEach) AS
total_sales
FROM products p JOIN orderdetails od ON p.productCode = od.productCode
GROUP BY Product_Name ORDER BY total_sales DESC LIMIT 5;
```

Explanation: Multiplies quantity by price for every order line and sums them to get total revenue per product.
The LIMIT 5 ensures only the most profitable items are displayed.

Approach 2: CTE for Revenue Calculation

```
WITH cte_total_sales AS (
   SELECT p.productName AS Product_Name, SUM(od.quantityOrdered * od.priceEach)
AS total_sales
   FROM products p JOIN orderdetails od ON p.productCode = od.productCode
```

```
    GROUP BY Product_Name
)
SELECT Product_Name, total_sales FROM cte_total_sales
ORDER BY total_sales DESC LIMIT 5;
```

Explanation: Similar to approach 1 but structured for scalability by placing core logic in a CTE.

This allows for easier refinement, such as calculating percentage of total sales from these top products.

#	Product_Name	total_sales
1	1992 Ferrari 360 Spider red	276839.98
2	2001 Ferrari Enzo	190755.86
3	1952 Alpine Renault 1300	190017.96
4	2003 Harley-Davidson Eagle Drag ...	170686.00
5	1968 Ford Mustang	161531.48

13. Identify customers who have placed orders but never received them (i.e., shippedDate IS NULL).

Approach 1: Distinct Join with NULL check

```
SELECT DISTINCT c.customerNumber, c.customerName
FROM customers c JOIN orders o ON c.customerNumber = o.customerNumber
WHERE o.shippedDate IS NULL;
```

Explanation: Simply joins customers to orders and filters for rows where the shipping date has not been recorded.

DISTINCT ensures each customer is listed only once even if they have multiple pending orders.

Approach 2: EXISTS Clause

```
SELECT c.customerNumber, c.customerName
FROM customers c
WHERE EXISTS (
    SELECT 1 FROM orders o WHERE o.customerNumber = c.customerNumber AND
```

```
o.shippedDate IS NULL
);
```

Explanation: Uses semi-join logic to check for the presence of at least one unshipped order per customer.
This is often more efficient as the database stops searching once it finds a single matching record.

#	customerNumber	customerName
1	448	Scandinavian Gift Ideas
2	131	Land of Toys Inc.
3	357	GiftsForHim.com
4	141	Euro+ Shopping Channel
5	144	Volvo Model Replicas, Co
6	328	Tekni Collectables Inc.
7	450	The Sharp Gifts Warehouse
8	362	Gifts4AllAges.com
9	282	Souveniers And Things Co.
10	124	Mini Gifts Distributors Ltd.
11	157	Diecast Classics Inc.

14. List employees who do not have a manager (reportsTo IS NULL) and also do not manage anyone.

Approach 1: NOT IN Subquery

```
SELECT employeeNumber, CONCAT(firstName, ' ', lastName) AS employee_name
FROM employees
WHERE reportsTo IS NULL AND employeeNumber NOT IN (
   SELECT reportsTo FROM employees WHERE reportsTo IS NOT NULL
);
```

Explanation: Filters for top-level staff (no manager) and then excludes anyone whose ID appears in the reportsTo column.
It effectively isolates employees who sit outside the traditional reporting chain.

Approach 2: NOT EXISTS Subquery

```
SELECT e1.employeeNumber, CONCAT(e1.firstName, ' ', e1.lastName) AS
employee_name
FROM employees e1
```

```
WHERE reportsTo IS NULL AND NOT EXISTS (
   SELECT 1 FROM employees e2 WHERE e1.employeeNumber = e2.reportsTo
);
```

Explanation: Uses the NOT EXISTS operator to verify that no other employee record references the current employee as manager.
This avoids potential pitfalls with NULL values that can occur with NOT IN in some database environments.

#	employeeNumber	employee_name

15. Identify the month with the highest number of orders placed in the dataset.

Approach 1: Group and Limit

```
SELECT MONTHNAME(orderDate) AS Month_Name, COUNT(orderNumber) AS Total_Orders
FROM orders GROUP BY Month_Name ORDER BY Total_Orders DESC LIMIT 1;
```

Explanation: Aggregates order counts by month name and sorts them to find the most frequent value.
Simple and effective for pinpointing peak seasonal demand.

Approach 2: HAVING with ALL Subquery

```
SELECT MONTHNAME(orderDate) AS M, COUNT(orderNumber) AS Total_Orders
FROM orders GROUP BY M
HAVING COUNT(orderNumber) >= ALL (
   SELECT COUNT(orderNumber) FROM orders GROUP BY MONTH(orderDate)
);
```

Explanation: Compares each month's total against a list of all monthly totals to find the maximum.

This mathematical approach handles ties automatically if multiple months share the exact same count.

#	Month_Name	Total_Orders
1	November	63

16. Write a query to find customers who have not placed any orders in the past 12 months.

```
SELECT c.customerNumber, c.customerName
FROM customers c JOIN orders o ON c.customerNumber = o.customerNumber
GROUP BY c.customerNumber, c.customerName
HAVING MAX(o.orderDate) < CURDATE() - INTERVAL 12 MONTH
ORDER BY c.customerNumber;
```

Explanation: Checks the most recent order date for each customer to see if it is older than one year.
Only customers who have historically ordered but have since stopped are included in this result.

Approach 2: WHERE NOT EXISTS

```
SELECT c.customerNumber, c.customerName
FROM customers c
WHERE NOT EXISTS (
   SELECT 1 FROM orders o
   WHERE o.customerNumber = c.customerNumber AND o.orderDate >= CURDATE() -
INTERVAL 12 MONTH
) ORDER BY c.customerNumber;
```

Explanation: Identifies customers who do not have any orders recorded within the last 12 month window.

This approach is comprehensive as it includes both lapsed customers and those who may have never ordered.

17. Identify all customer-employee pairs where the employee's office is in the same city as the customer.

#	customerNumber	customerName
1	103	Atelier graphique
2	112	Signal Gift Stores
3	114	Australian Collectors, Co.
4	119	La Rochelle Gifts
5	121	Baane Mini Imports
6	124	Mini Gifts Distributors Ltd.
7	128	Blauer See Auto, Co.
8	129	Mini Wheels Co.
9	131	Land of Toys Inc.
10	141	Euro+ Shopping Channel
11	144	Volvo Model Replicas, Co

Approach 1: Triple Table JOIN

```
SELECT c.customerName, CONCAT(e.firstName, ' ', e.lastName) AS employee_name,
c.city
FROM customers c
JOIN employees e ON c.salesRepEmployeeNumber = e.employeeNumber
JOIN offices o ON e.officeCode = o.officeCode
WHERE c.city = o.city;
```

Explanation: Connects customers to their assigned reps and those reps to their office locations.

Filtering for matching cities reveals opportunities for local account management and face-to face service.

Approach 2: JOIN with Subquery

```
SELECT c.customerName, CONCAT(e.firstName, ' ', e.lastName) AS employee_name,
c.city
FROM customers c
```

```
JOIN employees e ON c.salesRepEmployeeNumber = e.employeeNumber
WHERE c.city = (SELECT city FROM offices WHERE officeCode = e.officeCode);
```

Explanation: Uses a scalar subquery to find the office city for each employee during the filtering step.

This reduces main JOIN complexity while achieving the same geographical match.

#	customerName	employee_name	city
1	Mini Wheels Co.	Leslie Jennings	San Francisco
2	Corporate Gift Ideas Co.	Leslie Jennings	San Francisco
3	Diecast Collectables	Julie Firrelli	Boston
4	Gifts4AllAges.com	Steve Patterson	Boston
5	Muscle Machine Inc	Foon Yue Tseng	NYC
6	Vitachrome Inc.	Foon Yue Tseng	NYC
7	Classic Legends Inc.	Foon Yue Tseng	NYC
8	Microscale Inc.	Foon Yue Tseng	NYC
9	Land of Toys Inc.	George Vanauf	NYC
10	La Corne D'abondance, Co.	Loui Bondur	Paris
11	Lyon Souveniers	Loui Bondur	Paris

18. Calculate the total buyPrice of all products in stock and compare it to the total MSRP value.

Approach 1: Direct Summation

```
SELECT SUM(buyPrice * quantityInStock) AS total_buy_price,
SUM(MSRP * quantityInStock) AS total_msrp_value
FROM products;
```

Explanation: Calculates total cost of inventory versus potential market value if units were sold.

This provides a high-level view of current assets and potential profit margins.

No Alternate Approach Required

Standard aggregation is the optimal way to solve this valuation.

Explanation: Simple arithmetic summation across the products table is efficient. Used primarily for financial reporting and stock audits.

#	total_buy_price	total_msrp_value
1	30534316.23	56287967.27

19. Write a query to check if there are any circular reporting structures in the employees table.

Approach 1: Recursive Path Cycle Check

```
WITH RECURSIVE reporting_chain AS (
   SELECT employeeNumber, reportsTo, employeeNumber AS start_employee FROM
employees
   UNION ALL
   SELECT e.employeeNumber, e.reportsTo, rc.start_employee
   FROM employees e JOIN reporting_chain rc ON e.employeeNumber = rc.reportsTo
)
SELECT * FROM reporting_chain WHERE start_employee = reportsTo;
```

Explanation: Traces reporting paths recursively and alerts if an employee eventually reports back to themselves.

Essential for identifying data entry errors or logical flaws in organizational data.

Approach 2: Direct A-B-A Check

```
SELECT e1.employeeNumber AS emp1, e2.employeeNumber AS emp2
FROM employees e1
JOIN employees e2 ON e1.reportsTo = e2.employeeNumber AND e2.reportsTo =
e1.employeeNumber;
```

Explanation: Specifically looks for 'mutually reporting' pairs where two employees report directly to each other.

A simpler, faster check for the most common type of hierarchy error.

20. Write a query to identify employees who have not completed any sales in the past 6 months.

Approach 1: LEFT JOIN with Aggregation

```
SELECT e.employeeNumber, CONCAT(e.firstName, ' ', e.lastName) AS EmployeeName
FROM employees e
LEFT JOIN customers c ON c.salesRepEmployeeNumber = e.employeeNumber
LEFT JOIN orders o ON c.customerNumber = o.customerNumber
GROUP BY e.employeeNumber
HAVING MAX(o.orderDate) IS NULL OR MAX(o.orderDate) < CURDATE() -
INTERVAL 6 MONTH;
```

Explanation: Analyzes order dates per rep and identifies those whose last sale was over half a year ago.
The IS NULL condition ensures reps who have never made a sale are also flagged.

Approach 2: WHERE NOT EXISTS Semi-join

```
SELECT e.employeeNumber, CONCAT(e.firstName, ' ', e.lastName) AS EmployeeName
FROM employees e
WHERE NOT EXISTS (
   SELECT 1 FROM customers c
   JOIN orders o ON c.customerNumber = o.customerNumber
   WHERE c.salesRepEmployeeNumber = e.employeeNumber AND o.orderDate >=
CURDATE() - INTERVAL 6 MONTH
);
```

Explanation: Excludes any employee who has a recorded order within the last 6 months. This logic is highly effective for generating performance reports for HR or sales managers.

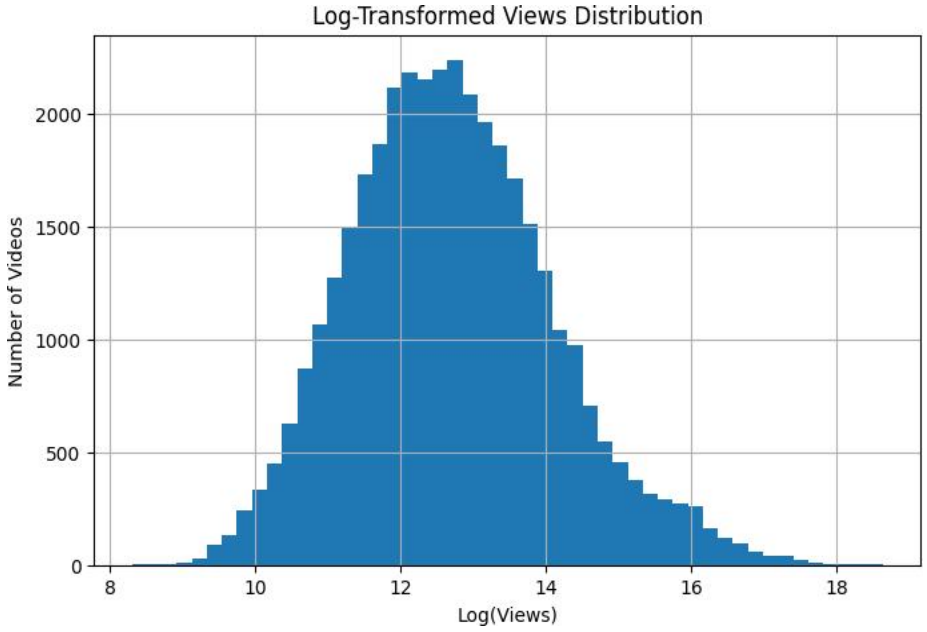
#	employeeNumber	EmployeeName
1	1002	Diane Murphy
2	1056	Mary Patterson
3	1076	Jeff Firrelli
4	1088	William Patterson
5	1102	Gerard Bondur
6	1143	Anthony Bow
7	1165	Leslie Jennings
8	1166	Leslie Thompson
9	1188	Julie Firrelli
10	1216	Steve Patterson
11	1286	Foon Yue Tseng

Python ETL and EDA on YouTube Dataset

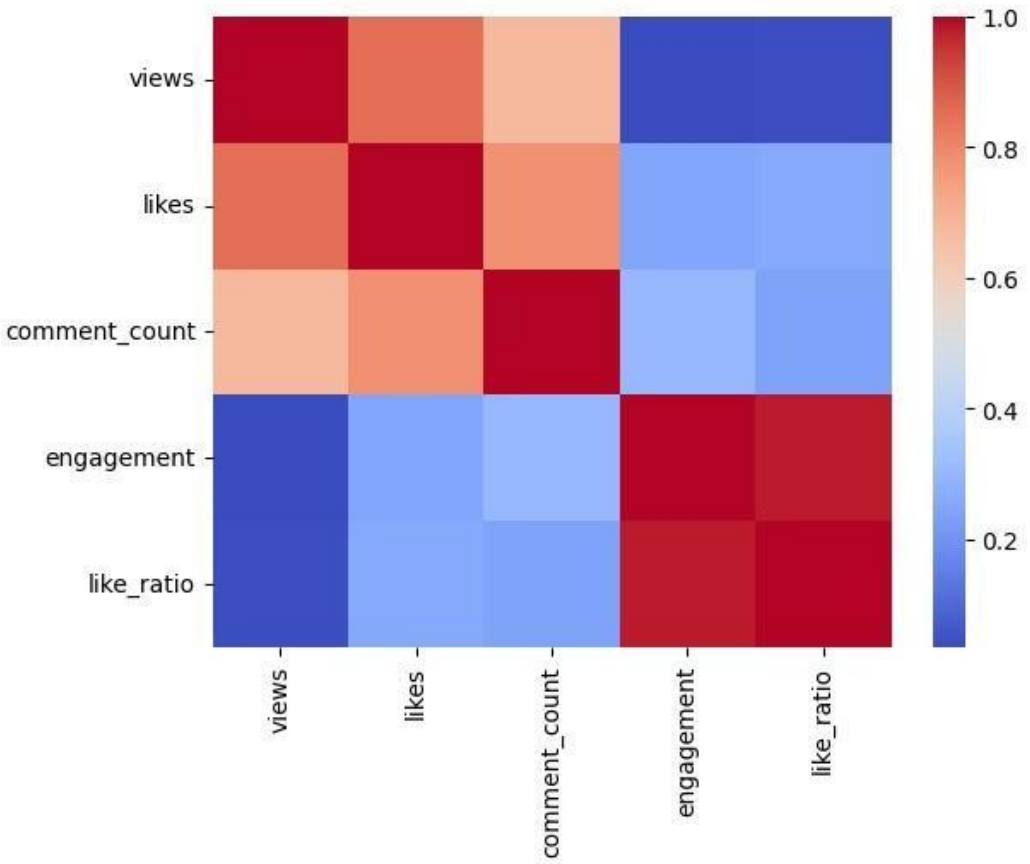
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from datetime import datetime as dt

# Load dataset
df = pd.read_csv(r'/home/nineleaps/Documents/ETL/archive (5)/INvideos.csv')

# Initial exploration
print(df.head())
print(df.columns)
print(df.dtypes)

# Convert date columns
df['publish_time'] = pd.to_datetime(df['publish_time'])
df['trending_date'] = pd.to_datetime(df['trending_date'], format="%y.%d.%m")

# Handle missing values
print(df.isnull().sum())
df['description'] = df['description'].fillna("Unknown")
```

```
# Feature Engineering
df['publish_time'] = df['publish_time'].dt.tz_localize(None)

# Days to trend
df['days_to_trend'] = (df['trending_date'] - df['publish_time']).dt.days

# Like ratio
df['like_ratio'] = df['likes'] / df['views']

# Engagement
df['engagement'] = (df['likes'] + df['comment_count']) / df['views']

# Time-based features
df['publish_hour'] = df['publish_time'].dt.hour
df['publish_day'] = df['publish_time'].dt.day_name()
df['publish_month'] = df['publish_time'].dt.month
df['is_weekend'] = df['publish_time'].dt.weekday >= 5

# Save cleaned dataset
df.to_csv('/home/nineleaps/Documents/ETL/Yt_cleaned.csv', index=False)

# Reload cleaned dataset
df = pd.read_csv('/home/nineleaps/Documents/ETL/Yt_cleaned.csv')

# Log transformation
df['log_views'] = np.log1p(df['views'])

# Histogram
plt.figure(figsize=(8,5))
df['log_views'].hist(bins=50)
```

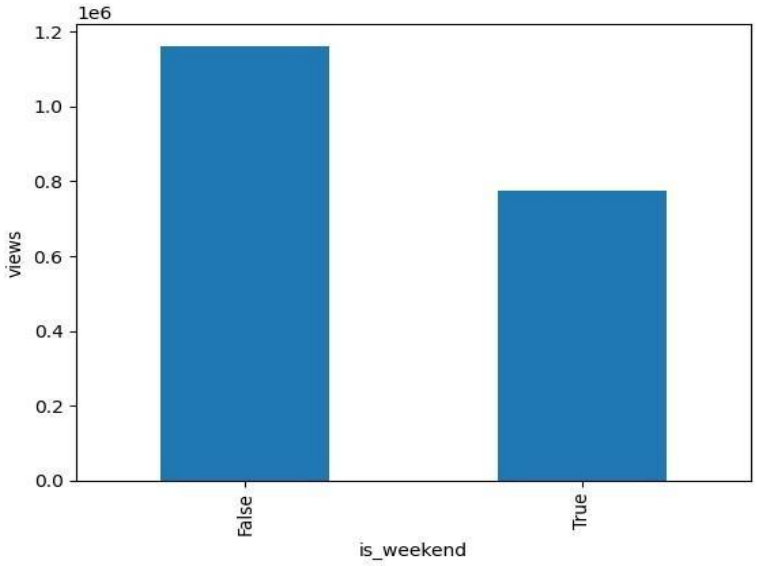
```
plt.title("Log-Transformed Views Distribution")
plt.xlabel("Log(Views)")
plt.ylabel("Number of Videos")
plt.show()
```



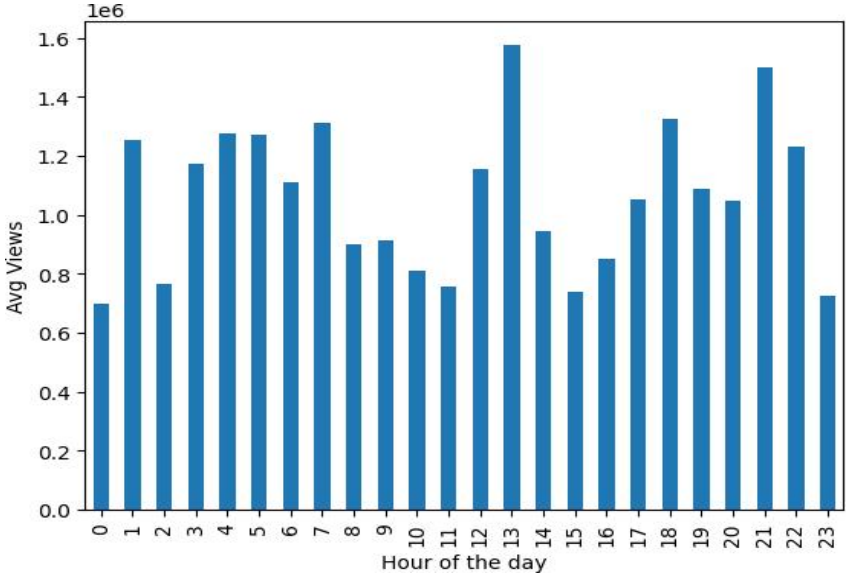

```
# Correlation matrix
corr = df[['views','likes','comment_count','engagement','like_ratio']].corr()
print(corr)

# Heatmap
sns.heatmap(corr, cmap='coolwarm')
plt.show()
```




```
# Weekend vs views
df.groupby('is_weekend')['views'].mean().plot(kind='bar')
plt.xlabel('is_weekend')
plt.ylabel('views')
plt.show()
```

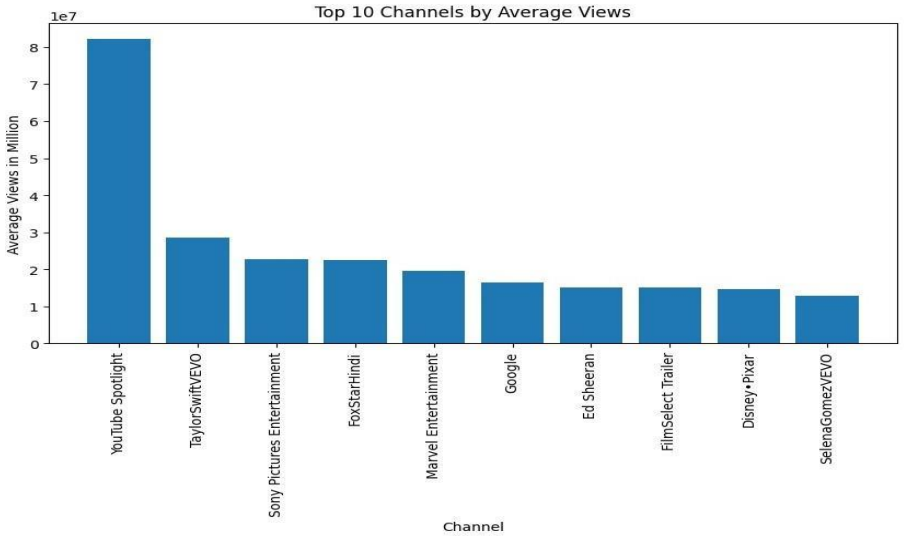


```
# Hour vs views
df.groupby('publish_hour')['views'].mean().plot(kind='bar')
plt.xlabel('Hour of the day')
plt.ylabel("Avg Views")
plt.show()
```

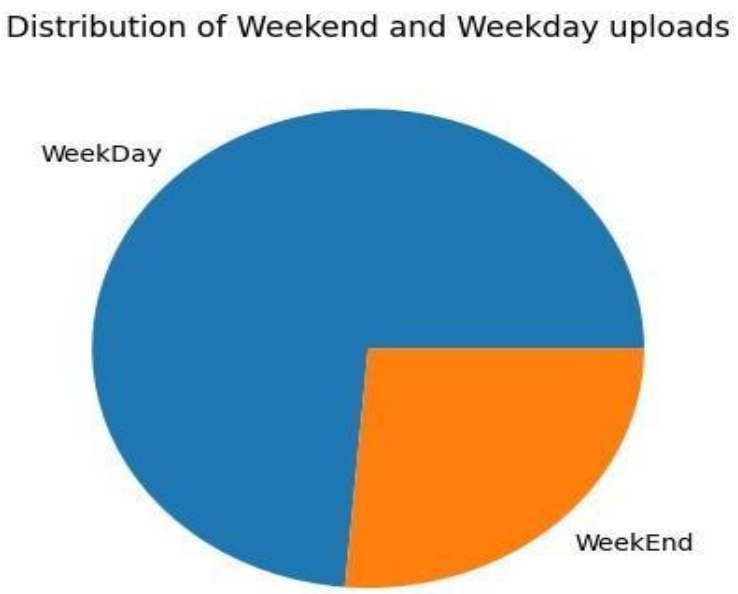



```
# Day vs views
df.groupby('publish_day')['views'].mean().plot(kind='bar')
plt.xlabel('Day of Week')
plt.ylabel("Avg Views")
plt.show()

# Top 10 Channel average views
channel_avg = (
    df.groupby('channel_title')['views']
    .mean()
    .reset_index()
    .rename(columns={'views': 'avg_views'})
```

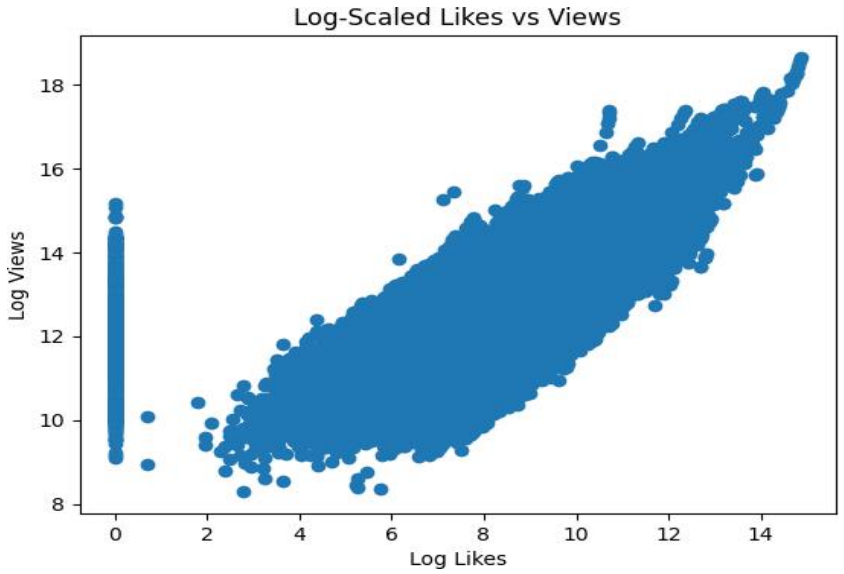
```
)

channel_avg = channel_avg.sort_values("avg_views", ascending=False)
top_channels = channel_avg.head(10)

plt.figure(figsize=(10,5))
plt.bar(top_channels['channel_title'], top_channels['avg_views'])
plt.xticks(rotation=90)
plt.title("Top 10 Channels by Average Views")
plt.xlabel("Channel")
plt.ylabel("Average Views")
plt.show()
```




```
# Weekend vs weekday pie chart
df['is_weekend'].value_counts().plot(kind='pie', labels=['WeekDay', 'WeekEnd'])
plt.title('Distribution of Weekend and Weekday uploads')
plt.show()
```

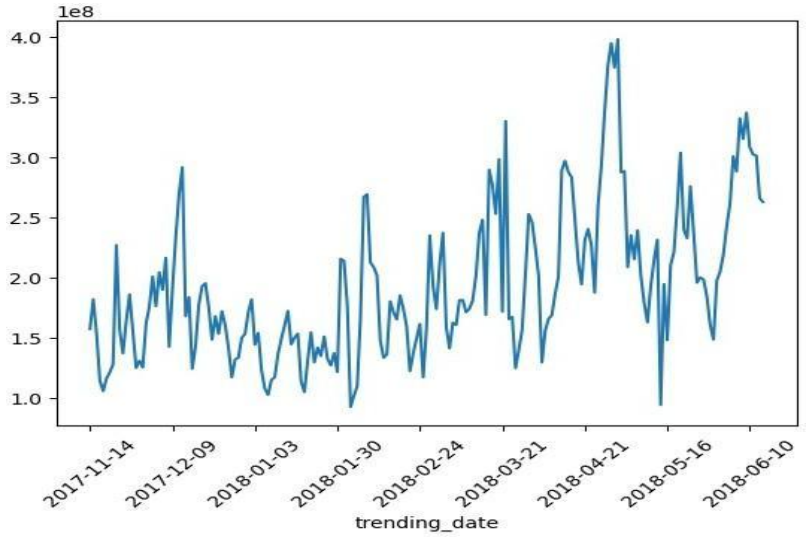



```
# Scatter plot (Likes vs Views)
plt.scatter(np.log1p(df['likes']), np.log1p(df['views']))
plt.xlabel("Log Likes")
plt.ylabel("Log Views")
plt.title("Log-Scaled Likes vs Views")
plt.show()
```




```
# Trending date vs total views
view_trend = df.groupby('trending_date')['views'].sum()
view_trend.plot(kind='line')
plt.xticks(rotation=45)
```

plt.show()


1e8
4.0
3.5
3.0
2.5
2.0
1.5
1.0
2017-11-14
2017-12-09
2018-01-03
2018-01-30
2018-02-24
2018-03-21
2018-04-21
2018-05-16
2018-06-10
trending_date